Integrated autonomous controller

Original code / PDF reference copy

SOURCE INTEGRITY

Original filename: Pasted text(9).txt

Source lines: 2,995 | Source bytes: 116,365

SHA-256 (original bytes):

`17946328c71838900b53a7bd0b00c42e108eed12c85cffc4577815e22ecd8b15`

The listing is a source-code record, not a claim of physical hardware validation.

ORIGINAL LINES 1-39

```
#include <Wire.h>
#include <math.h>
#include <HardwareSerial.h>
#include <stdio.h>
HardwareSerial DebugSerial(PA10, PA9);
// Raspberry Pi vision UART:
// RX=PB11, TX=PB10
// Physical wiring:
// Pi TX (GPIO14 / pin 8)  -> STM32 PB11 (RX3)
// Pi RX (GPIO15 / pin 10) -> STM32 PB10 (TX3)
// Pi GND                  -> STM32 GND
```

Integrated autonomous controller

```
// Packet:
// V,<steerPct>,<speedPct>,<confidencePct>\n
// Example:
// V,35,80,92
// = look ahead RIGHT 35%, request 80% speed,
//   with 92% path confidence.
HardwareSerial PiSerial(PB11, PB10);
// ======================================================
// ROBO RUMBLE - INTEGRATED AUTONOMOUS CONTROLLER
// FIXED I2C VERSION
// ======================================================
// ======================================================
// MOTOR DRIVER
// ======================================================
#define PWMA  PA8
#define AIN1  PB12
#define AIN2  PB13
#define PWMB  PB0
#define BIN1  PB14
#define BIN2  PB15
#define STBY  PB1
// ======================================================
// EMERGENCY STOP INPUT
// ======================================================
// Wokwi simulation:
// PB5 uses INPUT_PULLUP.
// Button released = HIGH
```

Integrated autonomous controller

```
39  // Button pressed  = LOW
```

ORIGINAL LINES 40-78

```
40  // IMPORTANT FOR THE PHYSICAL ROBOT:
41  // The real mushroom E-Stop must ALSO interrupt motor
42  // power directly. The GPIO input is only for status/
43  // software awareness; it is not the primary safety cut.
44  // ======================================================
45  #define ESTOP_PIN  PB5
46  // ======================================================
47  // MAIN ON/OFF SWITCH INPUT
48  // ======================================================
49  // Wokwi simulation:
50  // Use a LATCHING slide/rocker switch, not a momentary button.
51  // PA15 uses INPUT_PULLUP:
52  //   ON  = switch connects PA15 to GND -> LOW
53  //   OFF = switch open               -> HIGH
54  // A broken/open signal is therefore interpreted as OFF.
55  // IMPORTANT FOR THE PHYSICAL ROBOT:
56  // The real accessible main ON/OFF switch should interrupt
57  // the robot's main electrical supply. This GPIO is only a
58  // Wokwi simulation/status input. The E-Stop remains separate.
59  // ======================================================
60  #define MAIN_SWITCH_PIN  PA15
61  // ======================================================
62  // ENCODERS
63  // ======================================================
64  #define LEFT_ENC_A   PB8
```

Integrated autonomous controller

```
#define LEFT_ENC_B   PB9
// PB10/PB11 are reserved for Raspberry Pi UART.
// Right encoder uses PA11/PA12.
#define RIGHT_ENC_A  PA11
#define RIGHT_ENC_B  PA12
// ======================================================
// VL53L1X FRONT COLLISION SENSORS
// ======================================================
// The two ToF sensors are retained because other race cars
// and track walls can become collision hazards.
// Shared I2C:
//   SCL -> PB6
//   SDA -> PB7
// Individual XSHUT:
```

ORIGINAL LINES 79-117

```
//   Left  -> PB3
//   Right -> PB4
// Wokwi behavioural addresses:
//   Left  -> 0x30
//   Right -> 0x31
// NOTE FOR PHYSICAL HARDWARE:
// Real VL53L1X modules normally boot at the same default
// address. The final physical firmware must bring them up
// one at a time with XSHUT and assign unique addresses.
// ======================================================
#define LEFT_TOF_ADDR   0x30
#define RIGHT_TOF_ADDR  0x31
```

Integrated autonomous controller

```
#define LEFT_XSHUT      PB3
#define RIGHT_XSHUT     PB4
#define SIM_RANGE_REGISTER 0x0096
// ======================================================
// MPU6050
// ======================================================
#define MPU_ADDR          0x68
#define MPU_PWR_MGMT_1    0x6B
#define MPU_GYRO_CONFIG   0x1B
#define MPU_ACCEL_XOUT_H  0x3B
#define MPU_GYRO_ZOUT_H   0x47
#define MPU_WHO_AM_I      0x75
const float GYRO_SENSITIVITY = 131.0f;
// ======================================================
// IR ARRAY
// ======================================================
const uint8_t IR_PINS[8] = {
  PA0, PA1, PA2, PA3, PA4, PA5, PA6, PA7
};
// ======================================================
// ROAD-BOUNDARY INTERPRETATION
// ======================================================
// IMPORTANT:
// HIGH now means an IR sensor is seeing a ROAD BOUNDARY,
// not a centre line.
// Sensor layout:
//   IR1 IR2 IR3 IR4 | IR5 IR6 IR7 IR8
```

ORIGINAL LINES 118-156

Integrated autonomous controller

```
//   LEFT SIDE        |        RIGHT SIDE
// The weights are deliberately strongest near the
// centre of the robot. If a boundary reaches IR4 or IR5,
// the robot is close to crossing it and must turn away
// aggressively.
// Positive value  -> steer RIGHT (boundary on the left)
// Negative value  -> steer LEFT  (boundary on the right)
// Example:
//   00000000 -> road clear, drive straight
//   10000000 -> left outer boundary, gentle right
//   00100000 -> left inner boundary, stronger right
//   00000100 -> right inner boundary, stronger left
//   00000001 -> right outer boundary, gentle left
// ====================================================
const float BOUNDARY_STEER_WEIGHTS[8] = {
   1.0f, 2.0f, 3.0f, 4.0f, -4.0f, -3.0f, -2.0f, -1.0f
};
uint8_t sensor[8];
// ====================================================
// ENVIRONMENT + SPEED PROFILES
// ====================================================
// WOKWI:
//   RUNNING_IN_WOKWI = true
//   Keeps the already-tested ~38 RPM scaled motor model.
// PHYSICAL ROBOT:
//   RUNNING_IN_WOKWI = false
//   Start with:
```

Integrated autonomous controller

```
//     PHYSICAL_INITIAL_TEST_PROFILE = true
//     Normal target = 600 RPM ~= 1.35 m/s with 43 mm wheels.
//   After real traction, braking, current and PID testing:
//     PHYSICAL_INITIAL_TEST_PROFILE = false
//     Normal straight target = 1050 RPM ~= 2.36 m/s.
// The faster profile is NOT permission to drive every corner
// at 2.36 m/s. Camera speed recommendation still reduces the
// target before curves, while IR/ToF/IMU safety can override it.
// ======================================================
const bool RUNNING_IN_WOKWI = true;
// Used only when RUNNING_IN_WOKWI = false.
const bool PHYSICAL_INITIAL_TEST_PROFILE = true;
```

ORIGINAL LINES 157-195

```

// -------------------------
// WOKWI PROFILE
// -------------------------
const float WOKWI_NORMAL_BASE_RPM = 38.0f;
const float WOKWI_CAUTION_BASE_RPM = 28.0f;
const float WOKWI_SENSOR_FAULT_RPM = 20.0f;
const float WOKWI_BOUNDARY_CAUTION_RPM = 32.0f;
const float WOKWI_BOUNDARY_INNER_RPM = 28.0f;
const float WOKWI_BOUNDARY_CENTRE_RPM = 20.0f;
const float WOKWI_SKID_BASE_RPM = 22.0f;
const float WOKWI_MIN_TARGET_RPM = 8.0f;
const float WOKWI_MAX_TARGET_RPM = 40.0f;
// -------------------------
// PHYSICAL INITIAL TEST PROFILE
```

Integrated autonomous controller

```
// -------------------------
// 43 mm wheels:
//   450 RPM ~= 1.01 m/s
//   600 RPM ~= 1.35 m/s
// -------------------------
const float PHYS_TEST_NORMAL_BASE_RPM = 600.0f;
const float PHYS_TEST_CAUTION_BASE_RPM = 450.0f;
const float PHYS_TEST_SENSOR_FAULT_RPM = 350.0f;
const float PHYS_TEST_BOUNDARY_CAUTION_RPM = 500.0f;
const float PHYS_TEST_BOUNDARY_INNER_RPM = 420.0f;
const float PHYS_TEST_BOUNDARY_CENTRE_RPM = 320.0f;
const float PHYS_TEST_SKID_BASE_RPM = 350.0f;
const float PHYS_TEST_MIN_TARGET_RPM = 120.0f;
const float PHYS_TEST_MAX_TARGET_RPM = 650.0f;
// -------------------------
// PHYSICAL RACE PROFILE
// -------------------------
// 43 mm wheels:
//   450 RPM  ~= 1.01 m/s
//   525 RPM  ~= 1.18 m/s
//   735 RPM  ~= 1.66 m/s
//   1050 RPM ~= 2.36 m/s
// 1050 RPM is the first high-performance straight target.
// Speeds toward 3 m/s remain a later physical-test goal,
// not a default continuous race speed.
```

ORIGINAL LINES 196-230

```
// -------------------------
```

Integrated autonomous controller

```
const float PHYS_RACE_NORMAL_BASE_RPM = 1050.0f;
const float PHYS_RACE_CAUTION_BASE_RPM = 700.0f;
const float PHYS_RACE_SENSOR_FAULT_RPM = 500.0f;
const float PHYS_RACE_BOUNDARY_CAUTION_RPM = 800.0f;
const float PHYS_RACE_BOUNDARY_INNER_RPM = 650.0f;
const float PHYS_RACE_BOUNDARY_CENTRE_RPM = 500.0f;
const float PHYS_RACE_SKID_BASE_RPM = 500.0f;
const float PHYS_RACE_MIN_TARGET_RPM = 180.0f;
// Allows differential steering margin above the 1050 RPM base.
const float PHYS_RACE_MAX_TARGET_RPM = 1150.0f;
// -------------------------
// ACTIVE PROFILE
// -------------------------
const float NORMAL_BASE_RPM = RUNNING_IN_WOKWI ? WOKWI_NORMAL_BASE_RPM : ( PHYSICAL_INITIAL_TEST_PROFILE ? PHYS_TEST_NORMAL_BASE_RPM
          : PHYS_RACE_NORMAL_BASE_RPM );
const float CAUTION_BASE_RPM = RUNNING_IN_WOKWI ? WOKWI_CAUTION_BASE_RPM : ( PHYSICAL_INITIAL_TEST_PROFILE ? PHYS_TEST_CAUTION_BASE_RP M

          : PHYS_RACE_CAUTION_BASE_RPM );
const float SENSOR_FAULT_RPM = RUNNING_IN_WOKWI ? WOKWI_SENSOR_FAULT_RPM : ( PHYSICAL_INITIAL_TEST_PROFILE ? PHYS_TEST_SENSOR_FAULT_RP M

          : PHYS_RACE_SENSOR_FAULT_RPM );
const float BOUNDARY_CAUTION_RPM = RUNNING_IN_WOKWI ? WOKWI_BOUNDARY_CAUTION_RPM : ( PHYSICAL_INITIAL_TEST_PROFILE
          ? PHYS_TEST_BOUNDARY_CAUTION_RPM : PHYS_RACE_BOUNDARY_CAUTION_RPM );
const float BOUNDARY_INNER_RPM = RUNNING_IN_WOKWI ? WOKWI_BOUNDARY_INNER_RPM : ( PHYSICAL_INITIAL_TEST_PROFILE ? PHYS_TEST_BOUNDARY_IN
NER_RPM
          : PHYS_RACE_BOUNDARY_INNER_RPM );
const float BOUNDARY_CENTRE_RPM = RUNNING_IN_WOKWI ? WOKWI_BOUNDARY_CENTRE_RPM : ( PHYSICAL_INITIAL_TEST_PROFILE ? PHYS_TEST_BOUNDARY_
CENTRE_RPM

          : PHYS_RACE_BOUNDARY_CENTRE_RPM );
```

Integrated autonomous controller

```
const float SKID_BASE_RPM = RUNNING_IN_WOKWI ? WOKWI_SKID_BASE_RPM : ( PHYSICAL_INITIAL_TEST_PROFILE ? PHYS_TEST_SKID_BASE_RPM
          : PHYS_RACE_SKID_BASE_RPM );
const float MIN_TARGET_RPM = RUNNING_IN_WOKWI ? WOKWI_MIN_TARGET_RPM : ( PHYSICAL_INITIAL_TEST_PROFILE ? PHYS_TEST_MIN_TARGET_RPM
          : PHYS_RACE_MIN_TARGET_RPM );
const float MAX_TARGET_RPM = RUNNING_IN_WOKWI ? WOKWI_MAX_TARGET_RPM : ( PHYSICAL_INITIAL_TEST_PROFILE ? PHYS_TEST_MAX_TARGET_RPM
          : PHYS_RACE_MAX_TARGET_RPM );
// Steering gain is scaled to the active RPM range.
const float K_BOUNDARY_STEER = RUNNING_IN_WOKWI ? 3.0f : ( PHYSICAL_INITIAL_TEST_PROFILE ? 45.0f : 75.0f );
// ====================================================
```

ORIGINAL LINES 231-269

```

// RACE CONTROL TIMING
// ====================================================
// Boundary control is deliberately much faster than the
// motor-speed PID. This lets the robot react to the road
// edge quickly while the PID handles wheel-speed control.
// Wokwi:
//   Road/boundary loop = 10 ms  (100 Hz)
//   Motor PID loop     = 100 ms (10 Hz)
// On the physical STM32, the IR/boundary loop can later
// be pushed much faster after testing.
// ====================================================
const unsigned long ROAD_CONTROL_INTERVAL_MS = 10;
unsigned long previousRoadControlTime = 0;
// Other race cars and track walls are treated as
// collision hazards, so the ToF safety layer stays active.
const bool OBSTACLE_AVOIDANCE_ENABLED = true;
// ====================================================
```

Integrated autonomous controller

```
// WOKWI STABLE INTEGRATED SENSOR MODE
// ====================================================
// Earlier subsystem tests already proved the MPU6050,
// VL53L1X and INA219 logic individually. In the large
// integrated Wokwi build, several I2C devices/custom chips
// on one simulated bus can occasionally lock the simulator.
// This switch keeps the REAL I2C code in this same file,
// but uses deterministic internal sensor values while
// finishing the integrated racing/state-machine tests.
// WOKWI:
//   true  = use internal MPU/ToF/INA values
// PHYSICAL ROBOT:
//   false = use the real I2C sensors on PB6/PB7
// This changes NO physical wiring and does NOT remove the
// sensors from the final design/documentation.
// ====================================================
// ====================================================
// WOKWI SENSOR-SOURCE SELECTION
// ====================================================
// We want the VL53L1X DISTANCE SLIDERS to remain interactive,
// so ToF uses the real custom Wokwi I2C chips.
// The MPU6050 stays internally simulated for Wokwi stability.
```

ORIGINAL LINES 270-308

```
// The INA219 now uses the interactive Wokwi slider/custom I2C
// chip so current and voltage can be changed live during tests.
// PHYSICAL ROBOT:
// Set all three internal-simulation flags to false.
```

Integrated autonomous controller

```
// ====================================================
// FALSE = read the two custom VL53L1X chips and their sliders.
const bool USE_WOKWI_INTERNAL_TOF_SIM = false;
const bool USE_WOKWI_TOF_SLIDER_CHIPS = RUNNING_IN_WOKWI;
// TRUE = use a deterministic gyro value in Wokwi.
const bool USE_WOKWI_INTERNAL_IMU_SIM = RUNNING_IN_WOKWI;
// FALSE = read the interactive INA219 Wokwi slider/custom I2C chip.
// On the physical robot this is also false, so the real INA219 is
// read over I2C at address 0x40.
const bool USE_WOKWI_INTERNAL_INA_SIM = false;
// Diagnostic/documentation flag for the current Wokwi build.
const bool USE_WOKWI_INA219_SLIDER_CHIP = RUNNING_IN_WOKWI;
// -------------------------
// INTERNAL IMU TEST VALUES
// -------------------------
const bool WOKWI_IMU_ONLINE = true;
const float WOKWI_GYRO_Z_DPS = 0.0f;
// ====================================================
// AUTONOMOUS BRIDGE HANDLING
// ====================================================
// Physical robot:
//   Pi Camera predicts the bridge/ramp ahead.
//   MPU6050 confirms chassis pitch.
//   STM32 selects bridge state and safe speed automatically.
// No human chooses a bridge state during the race.
// Wokwi:
// This dedicated test automatically creates one complete
// bridge profile after MAIN SWITCH is turned ON.
```

Integrated autonomous controller

```
// ====================================================
// Bridge control logic remains fully enabled.
// Only the automatic Wokwi bridge DEMO is disabled in this
// dedicated corner-speed test build because the bridge sequence
// has already been validated separately.
// FINAL INTEGRATED BUILD:
// Bridge logic remains active, but no scripted Wokwi bridge test runs.
```

ORIGINAL LINES 309-347

```
const bool WOKWI_AUTO_BRIDGE_TEST = false;
const float WOKWI_MANUAL_PITCH_DEG = 0.0f;
const int WOKWI_MANUAL_BRIDGE_CONFIDENCE_PCT = 0;
// Automatic Wokwi timeline, measured from first ON state.
const unsigned long BRIDGE_TEST_FLAT_MS       = 3000;
const unsigned long BRIDGE_TEST_APPROACH_MS   = 5000;
const unsigned long BRIDGE_TEST_CLIMB_MS      = 8000;
const unsigned long BRIDGE_TEST_CREST_MS      = 9500;
const unsigned long BRIDGE_TEST_DESCENT_MS    = 12500;
const unsigned long BRIDGE_TEST_EXIT_MS       = 14500;
// First-pass thresholds for a mild bridge slope.
// Retune these on the physical bridge after measuring
// real pitch noise and the actual bridge angle.
const int BRIDGE_CAMERA_ENTER_CONFIDENCE_PCT = 70;
const int BRIDGE_CAMERA_CLEAR_CONFIDENCE_PCT = 40;
const float BRIDGE_CLIMB_ENTER_PITCH_DEG = 2.0f;
const float BRIDGE_CREST_PITCH_DEG = 1.2f;
const float BRIDGE_DESCENT_ENTER_PITCH_DEG = -2.0f;
const float BRIDGE_FLAT_PITCH_DEG = 1.0f;
```

Integrated autonomous controller

```
const unsigned long BRIDGE_EXIT_CONFIRM_MS = 800;
// ======================================================
// BRIDGE SPEED PROFILE
// ======================================================
// Wokwi keeps the mild-slope values already tested.
// Physical initial:
//   normal 600
//   approach 540
//   climb 510
//   crest 480
//   descent 480
//   exit 540
// Physical race:
//   normal 1050
//   approach 900
//   climb 850
//   crest 800
//   descent 800
//   exit 900
// Final physical values must be tuned using the real bridge.
```

ORIGINAL LINES 348-382

```
// ======================================================
const float WOKWI_BRIDGE_APPROACH_RPM = 34.0f;
const float WOKWI_BRIDGE_CLIMB_RPM = 32.0f;
const float WOKWI_BRIDGE_CREST_RPM = 30.0f;
const float WOKWI_BRIDGE_DESCENT_RPM = 30.0f;
const float WOKWI_BRIDGE_EXIT_RPM = 34.0f;
```

Integrated autonomous controller

```
const float PHYS_TEST_BRIDGE_APPROACH_RPM = 540.0f;
const float PHYS_TEST_BRIDGE_CLIMB_RPM = 510.0f;
const float PHYS_TEST_BRIDGE_CREST_RPM = 480.0f;
const float PHYS_TEST_BRIDGE_DESCENT_RPM = 480.0f;
const float PHYS_TEST_BRIDGE_EXIT_RPM = 540.0f;
const float PHYS_RACE_BRIDGE_APPROACH_RPM = 900.0f;
const float PHYS_RACE_BRIDGE_CLIMB_RPM = 850.0f;
const float PHYS_RACE_BRIDGE_CREST_RPM = 800.0f;
const float PHYS_RACE_BRIDGE_DESCENT_RPM = 800.0f;
const float PHYS_RACE_BRIDGE_EXIT_RPM = 900.0f;
const float BRIDGE_APPROACH_RPM = RUNNING_IN_WOKWI ? WOKWI_BRIDGE_APPROACH_RPM : ( PHYSICAL_INITIAL_TEST_PROFILE ? PHYS_TEST_BRIDGE_AP
PROACH_RPM
          : PHYS_RACE_BRIDGE_APPROACH_RPM );
const float BRIDGE_CLIMB_RPM = RUNNING_IN_WOKWI ? WOKWI_BRIDGE_CLIMB_RPM : ( PHYSICAL_INITIAL_TEST_PROFILE ? PHYS_TEST_BRIDGE_CLIMB_RP M

          : PHYS_RACE_BRIDGE_CLIMB_RPM );
const float BRIDGE_CREST_RPM = RUNNING_IN_WOKWI ? WOKWI_BRIDGE_CREST_RPM : ( PHYSICAL_INITIAL_TEST_PROFILE ? PHYS_TEST_BRIDGE_CREST_RP M

          : PHYS_RACE_BRIDGE_CREST_RPM );
const float BRIDGE_DESCENT_RPM = RUNNING_IN_WOKWI ? WOKWI_BRIDGE_DESCENT_RPM : ( PHYSICAL_INITIAL_TEST_PROFILE ? PHYS_TEST_BRIDGE_DESC
ENT_RPM
          : PHYS_RACE_BRIDGE_DESCENT_RPM );
const float BRIDGE_EXIT_RPM = RUNNING_IN_WOKWI ? WOKWI_BRIDGE_EXIT_RPM : ( PHYSICAL_INITIAL_TEST_PROFILE ? PHYS_TEST_BRIDGE_EXIT_RPM
          : PHYS_RACE_BRIDGE_EXIT_RPM );
const float BRIDGE_APPROACH_STEER_RPM = RUNNING_IN_WOKWI ? 8.0f : ( PHYSICAL_INITIAL_TEST_PROFILE ? 120.0f : 200.0f );
const float BRIDGE_CLIMB_STEER_RPM = RUNNING_IN_WOKWI ? 7.0f : ( PHYSICAL_INITIAL_TEST_PROFILE ? 100.0f : 170.0f );
const float BRIDGE_CREST_STEER_RPM = RUNNING_IN_WOKWI ? 6.0f : ( PHYSICAL_INITIAL_TEST_PROFILE ? 90.0f : 150.0f );
const float BRIDGE_DESCENT_STEER_RPM = RUNNING_IN_WOKWI ? 6.0f : ( PHYSICAL_INITIAL_TEST_PROFILE ? 90.0f : 150.0f );
const float BRIDGE_EXIT_STEER_RPM = RUNNING_IN_WOKWI ? 8.0f : ( PHYSICAL_INITIAL_TEST_PROFILE ? 120.0f : 200.0f );
```

Integrated autonomous controller

```
379 float wokwiSimulatedPitchDeg = 0.0f;
380 int wokwiSimulatedBridgeConfidencePct = 0;
381 bool wokwiBridgeTestStarted = false;
382 unsigned long wokwiBridgeTestStartTime = 0;
```

ORIGINAL LINES 383-421

```
383 // -------------------------
384 // INTERNAL INA219 FALLBACK VALUES
385 // -------------------------
386 // These values are used only if USE_WOKWI_INTERNAL_INA_SIM
387 // is changed back to true. In the current Wokwi build, the
388 // sliders control voltage and current live over I2C.
389 const bool WOKWI_INA_ONLINE = true;
390 const float WOKWI_BUS_VOLTAGE_V = 5.0f;
391 const float WOKWI_MOTOR_CURRENT_MA = 500.0f;
392 // ====================================================
393 // RASPBERRY PI CAMERA / PREDICTIVE STEERING
394 // ====================================================
395 // Camera/Pi:
396 //   sees the road ahead, predicts the next bend,
397 //   and recommends steering + approach speed.
398 // IR array:
399 //   reacts to the boundary directly under the car.
400 // STM32:
401 //   remains the final authority for motors, PID,
402 //   skid protection, stall protection and E-Stop.
403 // Packet from Pi:
404 //   V,<steerPct>,<speedPct>,<confidencePct>,<bridgeConfidencePct>
```

Integrated autonomous controller

```
// steerPct:
//   -100 = hard LEFT
//      0 = straight
//   +100 = hard RIGHT
// speedPct:
//   50..100 recommended race speed
// confidencePct:
//   0..100 path-detection confidence
// ====================================================
const unsigned long VISION_TIMEOUT_MS = 250;
// ====================================================
// WOKWI INTERNAL CAMERA / PI SIMULATOR
// ====================================================
// Wokwi cannot reproduce the real Raspberry Pi Camera
// processing pipeline. To keep testing reliable, this mode
// injects the SAME steering/speed/confidence command that the
// physical Raspberry Pi will later send over UART.
```

ORIGINAL LINES 422-460

```
// Leave this TRUE while finishing the Wokwi controller.
// Set it FALSE on the physical robot so PB11 receives the
// real Raspberry Pi UART packets.
// FINAL INTEGRATED WOKWI VISION PLACEHOLDER:
// No scripted test is run automatically.
// The internal vision source starts neutral:
//   steering        = 0%
//   speed           = 100%
//   confidence      = 95%
```

Integrated autonomous controller

```
//   bridge confidence = 0%
//   corner severity = 0%
// Therefore, after the operator deliberately starts the race,
// the default simulated road is simply a clear STRAIGHT.
// The normal IR, ToF, IMU, INA219, encoder, PID, bridge,
// corner-speed, stall, skid, E-Stop and fail-safe logic remains
// fully integrated and will react only when its inputs require it.
// ====================================================
const bool USE_WOKWI_INTERNAL_VISION_SIM = RUNNING_IN_WOKWI;
// FINAL INTEGRATED BUILD:
// No scripted/automatic corner test is allowed to run.
const bool WOKWI_AUTO_CORNER_SPEED_TEST = false;
const int WOKWI_VISION_STEER_PCT = 0;
const int WOKWI_VISION_SPEED_PCT = 100;
const int WOKWI_VISION_CONFIDENCE_PCT = 95;
const int WOKWI_VISION_CORNER_SEVERITY_PCT = 0;
const bool WOKWI_VISION_ONLINE = true;
// Automatic corner-test timing.
// This test now has its OWN timer and does not depend on the
// bridge test at all.
//   0-3 s   : STRAIGHT
//   3-6 s   : GENTLE RIGHT
//   6-9 s   : MEDIUM LEFT
//   9-12 s  : SHARP RIGHT
//   12-15 s : VERY SHARP LEFT
//   >15 s   : STRAIGHT AGAIN
const unsigned long CORNER_TEST_GENTLE_MS = 3000;
const unsigned long CORNER_TEST_MEDIUM_MS = 6000;
```

Integrated autonomous controller

```
459 const unsigned long CORNER_TEST_SHARP_MS = 9000;
460 const unsigned long CORNER_TEST_VERY_SHARP_MS = 12000;
```

ORIGINAL LINES 461-499

```
461 const unsigned long CORNER_TEST_END_MS = 15000;
462 bool wokwiCornerTestStarted = false;
463 unsigned long wokwiCornerTestStartTime = 0;
464 const unsigned long WOKWI_VISION_INTERVAL_MS = 40;
465 unsigned long previousWokwiVisionTime = 0;
466 const int VISION_MIN_CONFIDENCE_PCT = 45;
467 const int VISION_MIN_SPEED_PCT = 50;
468 const int VISION_MAX_SPEED_PCT = 100;
469 const float VISION_MAX_STEER_RPM = RUNNING_IN_WOKWI ? 10.0f : ( PHYSICAL_INITIAL_TEST_PROFILE ? 150.0f : 250.0f );
470 // Minimum speed the camera is allowed to request for a
471 // predicted sharp corner.
472 // Physical lower limit ~450 RPM ~= 1.0 m/s with 43 mm wheels.
473 const float VISION_MIN_BASE_RPM = RUNNING_IN_WOKWI ? 20.0f : 450.0f;
474 // Speed used if the camera is offline/stale/low-confidence.
475 const float VISION_FALLBACK_RPM = RUNNING_IN_WOKWI ? 28.0f : ( PHYSICAL_INITIAL_TEST_PROFILE ? 450.0f : 550.0f );
476 // ====================================================
477 // DYNAMIC CORNER-SPEED CONTROL
478 // ====================================================
479 // Cornering force:
480 //        F = m * v^2 / R
481 // Because speed is squared, the car must slow down
482 // non-linearly as the upcoming road curvature increases.
483 // The Pi will estimate CORNER SEVERITY from upcoming road
484 // curvature. STM32 then applies an independent speed cap:
```

Integrated autonomous controller

```
// speedFactor = 1 / sqrt(1 + K * severity)
// where severity is 0.0..1.0.
// With K = 2:
//   severity 0%   -> 100% straight speed
//   severity 25%  -> ~82%
//   severity 50%  -> ~71%
//   severity 75%  -> ~63%
//   severity 100% -> ~58%
// In physical race mode with 1050 RPM straight speed this is
// roughly:
//   0%   -> 1050 RPM ~= 2.36 m/s
//   25%  -> 857 RPM  ~= 1.93 m/s
//   50%  -> 742 RPM  ~= 1.67 m/s
//   75%  -> 664 RPM  ~= 1.50 m/s
//   100% -> 606 RPM  ~= 1.36 m/s
```

ORIGINAL LINES 500-538

```
// The Pi's own speed recommendation can request an EVEN LOWER
// speed. STM32 always chooses the safer/lower of the two.
// ======================================================
const float CORNER_CURVATURE_GAIN = 2.0f;
int visionSteerPct = 0;
int visionSpeedPct = 100;
int visionConfidencePct = 0;
// 0..100 confidence that the camera sees a bridge/ramp ahead.
int visionBridgeConfidencePct = 0;
// 0 = straight road ahead, 100 = strongest expected corner.
// This is look-ahead geometry, not merely current steering angle.
```

Integrated autonomous controller

```
int visionCornerSeverityPct = 0;
bool visionPacketValid = false;
unsigned long lastVisionPacketTime = 0;
char visionRxBuffer[48];
uint8_t visionRxIndex = 0;
// Pi UART diagnostics - visible in the NORMAL Serial Monitor.
unsigned long piRxByteCount = 0;
unsigned long piGoodPacketCount = 0;
unsigned long piBadPacketCount = 0;
char piLastRawPacket[48] = "(none)";
// ====================================================
// TOF / TTC
// ====================================================
// 20 Hz ToF/TTC update.
// A 500 ms interval is too slow for a racing robot.
const unsigned long TOF_INTERVAL_MS = 50;
const float MIN_CLOSING_SPEED_MM_S = 50.0f;
const float TTC_CRITICAL_S = 0.50f;
const float TTC_AVOID_S = 1.00f;
const float TTC_CAUTION_S = 2.00f;
const uint16_t CRITICAL_RANGE_MM = 180;
const uint16_t AVOID_RANGE_MM = 400;
const uint16_t CAUTION_RANGE_MM = 800;
const uint16_t SIDE_TOLERANCE_MM = 100;
// ====================================================
// INA219 CURRENT / POWER SENSOR
// ====================================================
```

Integrated autonomous controller

```
538 // Wokwi behavioural model:
```

ORIGINAL LINES 539-577

```
539 //   Address 0x40
540 //   Register 0x02 = bus voltage
541 //   Register 0x04 = current
542 // IMPORTANT:
543 // These current thresholds are SIMULATION STARTING VALUES.
544 // Tune them against the real motor stall-current measurements
545 // before using them on the physical robot.
546 // ====================================================
547 #define INA219_ADDR            0x40
548 #define INA219_REG_BUS_VOLTAGE 0x02
549 #define INA219_REG_CURRENT     0x04
550 const unsigned long INA_INTERVAL_MS = 100;
551 // Wokwi keeps the already-tested slider thresholds.
552 // Physical starting values are intentionally conservative and
553 // must be calibrated on the assembled car. The encoder-speed
554 // condition + 1 s confirmation + manual reset latch remain active.
555 const float HIGH_CURRENT_MA = RUNNING_IN_WOKWI ? 1200.0f : 500.0f;
556 const float STALL_CURRENT_MA = RUNNING_IN_WOKWI ? 1800.0f : 650.0f;
557 // Reference only. Confirmed stall uses MAIN OFF -> ON reset.
558 const float STALL_RESET_CURRENT_MA = RUNNING_IN_WOKWI ? 800.0f : 400.0f;
559 const float STALL_RPM_THRESHOLD = RUNNING_IN_WOKWI ? 5.0f : 80.0f;
560 const float STALL_COMMAND_RPM_THRESHOLD = RUNNING_IN_WOKWI ? 10.0f : 200.0f;
561 const unsigned long STALL_CONFIRM_MS = 1000;
562 // ====================================================
563 // ENCODERS
```

Integrated autonomous controller

```
// ======================================================
const float CPR = 280.0f;
// RPM sanity range.
// Wokwi: 42.86 RPM model -> 60 RPM sanity ceiling.
// Physical: 1500 RPM no-load motor -> 1800 RPM diagnostic ceiling.
const float MAX_VALID_RPM = RUNNING_IN_WOKWI ? 60.0f : 1800.0f;
const int8_t quadratureTable[16] = {
   0, +1, -1,  0, -1,  0,  0, +1, +1,  0,  0, -1, 0, -1, +1,  0
};
// ======================================================
// PID
// ======================================================
// Wokwi gains remain exactly as tested.
// Physical gains are safe STARTING VALUES only and require
```

ORIGINAL LINES 578-616

```

// real encoder/PWM tuning before competition-speed operation.
float Kp = RUNNING_IN_WOKWI ? 2.0f : 0.25f;
float Ki = RUNNING_IN_WOKWI ? 0.60f : 0.08f;
float Kd = RUNNING_IN_WOKWI ? 0.05f : 0.01f;
// 20 Hz wheel-speed PID for the high-speed Wokwi build.
// IR road-edge control remains faster at 100 Hz.
const unsigned long PID_INTERVAL_MS = 50;
// ======================================================
// PID STABILITY / OVERSPEED CONTROL
// ======================================================
// Wokwi N20 model calibration:
//   255 PWM -> about 42.86 RPM
```

Integrated autonomous controller

```
// We therefore use a speed-to-PWM feed-forward term.
// PID only trims the remaining error.
// Physical robot:
// Measure the real PWM-vs-RPM relationship and retune.
// ====================================================
const float WOKWI_MOTOR_MAX_RPM = 42.86f;
const float PHYSICAL_MOTOR_NO_LOAD_RPM = 1500.0f;
// Physical feed-forward is only a first approximation.
// Replace it with measured PWM-vs-loaded-RPM data.
const float PWM_FEEDFORWARD_PER_RPM = RUNNING_IN_WOKWI ? ( 255.0f / WOKWI_MOTOR_MAX_RPM ) : ( 255.0f / PHYSICAL_MOTOR_NO_LOAD_RPM );
// Integral anti-windup bound.
const float PID_INTEGRAL_LIMIT = RUNNING_IN_WOKWI ? 15.0f : 400.0f;
// Smooth encoder RPM samples before PID.
const float RPM_FILTER_ALPHA = 0.35f;
// Bleed positive integral if actual RPM exceeds setpoint.
const float OVERSPEED_MARGIN_RPM = RUNNING_IN_WOKWI ? 1.0f : 25.0f;
const float OVERSPEED_INTEGRAL_BLEED = 0.50f;
// ====================================================
// SPEED RAMP / SLEW-RATE LIMITER
// ====================================================
// Navigation can request a new wheel speed immediately,
// but the PID follows a ramped setpoint instead of jumping
// straight to the requested value.
// This reduces launch wheel-slip, current spikes, false
// stall detection, and violent weight transfer.
// Wokwi starting values:
//   acceleration = 30 RPM/s
```

ORIGINAL LINES 617-654

Integrated autonomous controller

```
//   deceleration = 120 RPM/s
// At the 50 ms PID interval this is about:
//   +1.5 RPM per PID step while accelerating
//   -6.0 RPM per PID step while decelerating
// Deceleration is deliberately faster so the robot can
// react quickly to corners and boundaries.
// IMPORTANT: these are simulation values. The physical
// 1500 RPM N20 robot must be retuned using measured grip,
// current draw, mass, bridge slope and wheel-slip.
// ====================================================
const float ACCEL_RAMP_RPM_PER_SEC = RUNNING_IN_WOKWI ? 30.0f : ( PHYSICAL_INITIAL_TEST_PROFILE ? 300.0f : 450.0f );
const float DECEL_RAMP_RPM_PER_SEC = RUNNING_IN_WOKWI ? 120.0f : ( PHYSICAL_INITIAL_TEST_PROFILE ? 450.0f : 700.0f );
// ====================================================
// SOFTWARE PWM
// ====================================================
const unsigned long PWM_PERIOD_US = 10000;
// ====================================================
// AVOIDANCE SPEED PROFILE
// ====================================================
const float AVOID_SLOW_RPM = RUNNING_IN_WOKWI ? 14.0f : ( PHYSICAL_INITIAL_TEST_PROFILE ? 220.0f : 400.0f );
const float AVOID_FAST_RPM = RUNNING_IN_WOKWI ? 34.0f : ( PHYSICAL_INITIAL_TEST_PROFILE ? 540.0f : 900.0f );
const float CRITICAL_AVOID_SLOW_RPM = RUNNING_IN_WOKWI ? 8.0f : ( PHYSICAL_INITIAL_TEST_PROFILE ? 120.0f : 180.0f );
const float CRITICAL_AVOID_FAST_RPM = RUNNING_IN_WOKWI ? 38.0f : ( PHYSICAL_INITIAL_TEST_PROFILE ? 600.0f : 1050.0f );
// ====================================================
// NAVIGATION STATES
// ====================================================
enum NavigationState {
```

Integrated autonomous controller

```
  STATE_EMERGENCY_STOP, STATE_VISION_DRIVE, STATE_VISION_FALLBACK, STATE_ROAD_DRIVE, STATE_BOUNDARY_CORRECT_LEFT, STATE_BOUNDARY_CORRECT_RIGHT,
  STATE_BOUNDARY_CAUTION, STATE_BOUNDARY_ESCAPE, STATE_CAUTION, STATE_AVOID_LEFT, STATE_AVOID_RIGHT, STATE_CRITICAL_AVOID_LEFT,
  STATE_CRITICAL_AVOID_RIGHT, STATE_CRITICAL_STOP,
  // Obstacle requires an avoidance turn, but the road
  // boundary blocks that escape direction.
  STATE_NO_SAFE_PATH, STATE_MOTOR_STALL, STATE_SKID_CAUTION, STATE_SENSOR_FAULT,
  // Appended so all previously validated state numbers stay unchanged.
  STATE_SYSTEM_OFF, STATE_BRIDGE_APPROACH, STATE_BRIDGE_CLIMB, STATE_BRIDGE_CREST, STATE_BRIDGE_DESCENT, STATE_BRIDGE_EXIT
};
// Safe boot: the race controller always starts OFF.
// It cannot drive until the MAIN switch has been observed OFF
```

ORIGINAL LINES 655-693

```

// and is then deliberately switched ON.
NavigationState currentState = STATE_SYSTEM_OFF;
NavigationState previousState = STATE_SYSTEM_OFF;
// Counts impossible/invalid state values detected at runtime.
// Any invalid state immediately falls back to CRITICAL STOP.
unsigned long invalidStateRecoveryCount = 0;
bool emergencyStopActive = false;
// ====================================================
// DEBOUNCED MAIN SWITCH + RACE-START INTERLOCK
// ====================================================
// mainSwitchRawOn:
//   immediate electrical reading from PA15.
// mainSwitchOn:
//   stable/debounced switch state used by the controller.
```

Integrated autonomous controller

```
// The robot ALWAYS boots with the race interlock locked.
// It must observe a stable MAIN OFF state, and only a later
// stable OFF -> ON transition is allowed to start the race.
// This also prevents one physical switch movement from being
// interpreted as several starts because of contact bounce.
// =======================================================
bool mainSwitchRawOn = false;
bool mainSwitchOn = false;
bool mainSwitchSeenOffSinceBoot = false;
bool raceStartInterlockCleared = false;
unsigned long mainSwitchLastRawChangeMs = 0;
// 60 ms is comfortably above normal mechanical switch bounce
// and is negligible compared with race setup time.
const unsigned long MAIN_SWITCH_DEBOUNCE_MS = 60;
// =======================================================
// ROAD / BOUNDARY DATA
// =======================================================
// boundarySteerError:
//   positive -> steer RIGHT
//   negative -> steer LEFT
// leftBoundarySeverity/rightBoundarySeverity:
//   0 = no boundary
//   1 = outer sensor
//   2 = next sensor
//   3 = inner sensor
```

ORIGINAL LINES 694-732

```
//   4 = centre-adjacent sensor
```

Integrated autonomous controller

```
// ====================================================
float boundarySteerError = 0.0f;
float lastBoundarySteerError = 0.0f;
float leftBoundarySeverity = 0.0f;
float rightBoundarySeverity = 0.0f;
float boundarySeverity = 0.0f;
bool boundaryDetected = false;
bool centreBoundaryDetected = false;
bool bothSidesBoundaryDetected = false;
int activeBoundarySensors = 0;
// ====================================================
// TOF DATA
// ====================================================
uint16_t leftDistance = 2000;
uint16_t rightDistance = 2000;
uint16_t previousLeftDistance = 2000;
uint16_t previousRightDistance = 2000;
float leftClosingSpeed = 0.0f;
float rightClosingSpeed = 0.0f;
float leftTTC = -1.0f;
float rightTTC = -1.0f;
bool tofValid = false;
bool previousToFValid = false;
unsigned long previousToFTime = 0;
// ====================================================
// INA219 DATA
// ====================================================
float busVoltageV = 0.0f;
```

Integrated autonomous controller

```
float motorCurrentMA = 0.0f;
bool inaValid = false;
bool motorStallDetected = false;
bool motorStallLatched = false;
// Bit 0 = left wheel, bit 1 = right wheel.
// These let us identify a one-wheel stall instead of averaging
// both wheel speeds together.
uint8_t stallCandidateMask = 0;
uint8_t stallLatchedSideMask = 0;
unsigned long previousINATime = 0;
```

ORIGINAL LINES 733-771

```

unsigned long stallConditionStart = 0;
// ======================================================
// IMU
// ======================================================
float gyroZ = 0.0f;
// 0 = flat, positive = climbing, negative = descending.
float pitchDeg = 0.0f;
bool imuValid = false;
bool possibleSkid = false;
enum BridgePhase {
  BRIDGE_NONE, BRIDGE_APPROACH, BRIDGE_CLIMB, BRIDGE_CREST, BRIDGE_DESCENT, BRIDGE_EXIT
};
BridgePhase bridgePhase = BRIDGE_NONE;
float maximumBridgeClimbPitchDeg = 0.0f;
unsigned long bridgeExitStartTime = 0;
// ======================================================
```

Integrated autonomous controller

```
// ENCODER STATE
// ====================================================
long leftEncoderCount = 0;
long rightEncoderCount = 0;
long previousLeftPIDCount = 0;
long previousRightPIDCount = 0;
uint8_t previousLeftEncoderState = 0;
uint8_t previousRightEncoderState = 0;
// ====================================================
// RPM
// ====================================================
// Filtered encoder RPM used by the controller.
float leftRPM = 0.0f;
float rightRPM = 0.0f;
// Latest raw encoder RPM samples.
float leftRawRPM = 0.0f;
float rightRawRPM = 0.0f;
bool leftRPMFilterInitialised = false;
bool rightRPMFilterInitialised = false;
// True when the most recent encoder RPM sample was finite
// and inside the physically reasonable simulation range.
bool leftRPMValid = true;
bool rightRPMValid = true;
```

ORIGINAL LINES 772-810

```

// Navigation-requested wheel speeds.
float leftTargetRPM = NORMAL_BASE_RPM;
float rightTargetRPM = NORMAL_BASE_RPM;
```

Integrated autonomous controller

```
// PID setpoints after acceleration/deceleration limiting.
// Start at zero for a controlled soft launch.
float leftRampedTargetRPM = 0.0f;
float rightRampedTargetRPM = 0.0f;
// ================================================
// PID STATE
// ================================================
float leftIntegral = 0.0f;
float rightIntegral = 0.0f;
float leftPreviousError = 0.0f;
float rightPreviousError = 0.0f;
unsigned long previousPIDTime = 0;
// ================================================
// PWM STATE
// ================================================
// Start with zero motor drive. PID increases PWM only
// as the ramped target rises.
int leftPWM = 0;
int rightPWM = 0;
bool leftPWMState = false;
bool rightPWMState = false;
unsigned long pwmPeriodStart = 0;
// ================================================
// SERIAL TIMING
// ================================================
const unsigned long PRINT_INTERVAL_MS = 1500;
unsigned long previousPrintTime = 0;
// ================================================
```

Integrated autonomous controller

```
// ENCODERS
// ================================================
void readLeftEncoder() {
  uint8_t a = digitalRead(LEFT_ENC_A);
  uint8_t b = digitalRead(LEFT_ENC_B);
  uint8_t current = (a << 1) | b;
  if (current == previousLeftEncoderState) return;
  uint8_t index = (previousLeftEncoderState << 2) | current;
```

ORIGINAL LINES 811-849

```
  leftEncoderCount += quadratureTable[index];
  previousLeftEncoderState = current;
}
void readRightEncoder() {
  uint8_t a = digitalRead(RIGHT_ENC_A);
  uint8_t b = digitalRead(RIGHT_ENC_B);
  uint8_t current = (a << 1) | b;
  if (current == previousRightEncoderState) return;
  uint8_t index = (previousRightEncoderState << 2) | current;
  rightEncoderCount += quadratureTable[index];
  previousRightEncoderState = current;
}
// ================================================
// SOFTWARE PWM
// ================================================
void updateSoftwarePWM() {
  unsigned long now = micros();
  if ( now - pwmPeriodStart >= PWM_PERIOD_US ) {
```

Integrated autonomous controller

```
    pwmPeriodStart = now;
  }
  unsigned long position = now - pwmPeriodStart;
  // LEFT
  unsigned long leftHighTime = ((unsigned long)leftPWM * PWM_PERIOD_US) / 255UL;
  bool desiredLeft;
  if (leftPWM <= 0) desiredLeft = false;
  else if (leftPWM >= 255) desiredLeft = true;
  else desiredLeft = position < leftHighTime;
  if (desiredLeft != leftPWMState) {
    leftPWMState = desiredLeft;
    digitalWrite( PWMA, leftPWMState ? HIGH : LOW );
  }
  // RIGHT
  unsigned long rightHighTime = ((unsigned long)rightPWM * PWM_PERIOD_US) / 255UL;
  bool desiredRight;
  if (rightPWM <= 0) desiredRight = false;
  else if (rightPWM >= 255) desiredRight = true;
  else desiredRight = position < rightHighTime;
  if (desiredRight != rightPWMState) {
    rightPWMState = desiredRight;
```

ORIGINAL LINES 850-888

```
    digitalWrite( PWMB, rightPWMState ? HIGH : LOW );
  }
}
void serviceMotors() {
  updateSoftwarePWM();
```

Integrated autonomous controller

```
  readLeftEncoder();
  readRightEncoder();
}
// ========================================================
// MOTOR DIRECTION
// ========================================================
void motorsForward() {
  // Direction commands are prepared here. The motor
  // driver is enabled only while the E-Stop is released.
  digitalWrite( STBY, ( emergencyStopActive || !mainSwitchOn || !raceStartInterlockCleared ) ? LOW : HIGH );
  digitalWrite(AIN1, HIGH);
  digitalWrite(AIN2, LOW);
  digitalWrite(BIN1, HIGH);
  digitalWrite(BIN2, LOW);
}
// ========================================================
// EMERGENCY STOP
// ========================================================
void readEmergencyStop() {
  emergencyStopActive = ( digitalRead(ESTOP_PIN) == LOW );
}
void applyEmergencyStopImmediately() {
  // Do not wait for the 250 ms PID update.
  currentState = STATE_EMERGENCY_STOP;
  leftTargetRPM = 0.0f;
  rightTargetRPM = 0.0f;
  leftRampedTargetRPM = 0.0f;
  rightRampedTargetRPM = 0.0f;
```

Integrated autonomous controller

```
  leftPWM = 0;
  rightPWM = 0;
  leftPWMState = false;
  rightPWMState = false;
  digitalWrite(PWMA, LOW);
  digitalWrite(PWMB, LOW);
```

ORIGINAL LINES 889-927

```
  // Hardware-driver shutdown in the Wokwi controller.
  digitalWrite(STBY, LOW);
}
// ====================================================
// MAIN ON/OFF SWITCH
// ====================================================
void resetDriveMemoryForRaceStart() {
  // Start both motor channels from exactly the same neutral state.
  leftTargetRPM = 0.0f;
  rightTargetRPM = 0.0f;
  leftRampedTargetRPM = 0.0f;
  rightRampedTargetRPM = 0.0f;
  leftIntegral = 0.0f;
  rightIntegral = 0.0f;
  leftPreviousError = 0.0f;
  rightPreviousError = 0.0f;
  leftPWM = 0;
  rightPWM = 0;
  leftPWMState = false;
  rightPWMState = false;
```

Integrated autonomous controller

```
    // Neutral navigation memory.
    boundarySteerError = 0.0f;
    lastBoundarySteerError = 0.0f;
    // Neutral internal vision placeholder.
    visionSteerPct = 0;
    visionSpeedPct = 100;
    visionConfidencePct = 95;
    visionBridgeConfidencePct = 0;
    visionCornerSeverityPct = 0;
    // No scripted test is allowed to carry into the race.
    bridgePhase = BRIDGE_NONE;
    wokwiBridgeTestStarted = false;
    wokwiCornerTestStarted = false;
    // PID timing starts fresh so the first ramp step is symmetrical.
    previousPIDTime = millis();
    // Motor driver remains disabled until the main loop reaches
    // the normal enabled section.
    digitalWrite(PWMA, LOW);
    digitalWrite(PWMB, LOW);
```

ORIGINAL LINES 928-966

```
    digitalWrite(STBY, LOW);
}
void initialiseMainSwitch() {
    // INPUT_PULLUP:
    // LOW  = ON
    // HIGH = OFF
    mainSwitchRawOn = ( digitalRead(MAIN_SWITCH_PIN) == LOW );
```

Integrated autonomous controller

```
935   mainSwitchOn = mainSwitchRawOn;
936   mainSwitchLastRawChangeMs = millis();
937   // IMPORTANT:
938   // Never start a race merely because the switch happened to be
939   // ON when the MCU/Wokwi simulation booted.
940   raceStartInterlockCleared = false;
941   if (!mainSwitchOn) {
942     mainSwitchSeenOffSinceBoot = true;
943   }
944   else {
945     mainSwitchSeenOffSinceBoot = false;
946   }
947   // Safe startup outputs.
948   applySystemOffImmediately();
949 }
950 void readMainSwitch() {
951   bool rawOn = ( digitalRead(MAIN_SWITCH_PIN) == LOW );
952   // A raw transition only starts/restarts the debounce timer.
953   // It does NOT immediately change the controller state.
954   if (rawOn != mainSwitchRawOn) {
955     mainSwitchRawOn = rawOn;
956     mainSwitchLastRawChangeMs = millis();
957   }
958   // Wait until the raw state has remained unchanged long enough.
959   if ( (millis() - mainSwitchLastRawChangeMs) < MAIN_SWITCH_DEBOUNCE_MS ) {
960     return;
961   }
962   // No new stable switch transition.
```

Integrated autonomous controller

```
  if (mainSwitchRawOn == mainSwitchOn) {
    return;
  }
  // Accept exactly one stable transition.
```

ORIGINAL LINES 967-1,005

```
  bool oldStableOn = mainSwitchOn;
  mainSwitchOn = mainSwitchRawOn;
  // ====================================================
  // STABLE ON -> OFF
  // ====================================================
  if ( oldStableOn && !mainSwitchOn ) {
    // The robot is now deliberately OFF.
    raceStartInterlockCleared = false;
    mainSwitchSeenOffSinceBoot = true;
    // A confirmed motor stall may only be cleared by this
    // deliberate MAIN OFF action.
    resetMotorStallLatchWhileSystemOff();
    // Immediately make both motor channels identical and safe.
    applySystemOffImmediately();
    DebugSerial.println( "[POWER] MAIN SWITCH -> OFF : SYSTEM SAFE" );
    return;
  }
  // ====================================================
  // FIRST/STABLE OFF OBSERVATION
  // ====================================================
  if (!mainSwitchOn) {
    mainSwitchSeenOffSinceBoot = true;
```

Integrated autonomous controller

```
    raceStartInterlockCleared = false;
    applySystemOffImmediately();
    return;
  }
  // ================================================
  // STABLE OFF -> ON = ONE RACE START
  // ================================================
  if ( !oldStableOn && mainSwitchOn && mainSwitchSeenOffSinceBoot && !raceStartInterlockCleared ) {
    resetDriveMemoryForRaceStart();
    raceStartInterlockCleared = true;
    DebugSerial.println( "[START] MAIN OFF -> ON : RACE START ENABLED" );
    return;
  }
  // Any other situation remains locked for safety.
  raceStartInterlockCleared = false;
}
void applySystemOffImmediately() {
```

ORIGINAL LINES 1,006-1,044

```
  currentState = STATE_SYSTEM_OFF;
  leftTargetRPM = 0.0f;
  rightTargetRPM = 0.0f;
  leftRampedTargetRPM = 0.0f;
  rightRampedTargetRPM = 0.0f;
  leftPWM = 0;
  rightPWM = 0;
  leftPWMState = false;
  rightPWMState = false;
```

Integrated autonomous controller

```
  leftIntegral = 0.0f;
  rightIntegral = 0.0f;
  digitalWrite(PWMA, LOW);
  digitalWrite(PWMB, LOW);
  digitalWrite(STBY, LOW);
}
// ====================================================
// WOKWI AUTONOMOUS BRIDGE SCENARIO
// ====================================================
void updateWokwiBridgeScenario() {
  if (!USE_WOKWI_INTERNAL_IMU_SIM) {
    return;
  }
  if (!WOKWI_AUTO_BRIDGE_TEST) {
    wokwiSimulatedPitchDeg = WOKWI_MANUAL_PITCH_DEG;
    wokwiSimulatedBridgeConfidencePct = WOKWI_MANUAL_BRIDGE_CONFIDENCE_PCT;
    return;
  }
  if (!wokwiBridgeTestStarted) {
    wokwiBridgeTestStarted = true;
    wokwiBridgeTestStartTime = millis();
  }
  unsigned long elapsed = millis() - wokwiBridgeTestStartTime;
  if (elapsed < BRIDGE_TEST_FLAT_MS) {
    wokwiSimulatedPitchDeg = 0.0f;
    wokwiSimulatedBridgeConfidencePct = 0;
    return;
  }
```

Integrated autonomous controller

```
  if (elapsed < BRIDGE_TEST_APPROACH_MS) {
    wokwiSimulatedPitchDeg = 0.0f;
```

ORIGINAL LINES 1,045-1,083

```
    wokwiSimulatedBridgeConfidencePct = 90;
    return;
  }
  if (elapsed < BRIDGE_TEST_CLIMB_MS) {
    wokwiSimulatedPitchDeg = 4.5f;
    wokwiSimulatedBridgeConfidencePct = 95;
    return;
  }
  if (elapsed < BRIDGE_TEST_CREST_MS) {
    wokwiSimulatedPitchDeg = 1.0f;
    wokwiSimulatedBridgeConfidencePct = 95;
    return;
  }
  if (elapsed < BRIDGE_TEST_DESCENT_MS) {
    wokwiSimulatedPitchDeg = -4.5f;
    wokwiSimulatedBridgeConfidencePct = 90;
    return;
  }
  if (elapsed < BRIDGE_TEST_EXIT_MS) {
    wokwiSimulatedPitchDeg = 0.0f;
    wokwiSimulatedBridgeConfidencePct = 20;
    return;
  }
  wokwiSimulatedPitchDeg = 0.0f;
```

Integrated autonomous controller

```
  wokwiSimulatedBridgeConfidencePct = 0;
}
// ====================================================
// RASPBERRY PI VISION UART
// ====================================================
bool visionIsOnline() {
  if (!visionPacketValid) return false;
  return ( millis() - lastVisionPacketTime ) <= VISION_TIMEOUT_MS;
}
bool visionIsTrusted() {
  return visionIsOnline() && visionConfidencePct >= VISION_MIN_CONFIDENCE_PCT;
}
const char *cornerClassName() {
  if (visionCornerSeverityPct < 15) {
    return "STRAIGHT";
```

ORIGINAL LINES 1,084-1,122

```
  }
  if (visionCornerSeverityPct < 35) {
    return "GENTLE";
  }
  if (visionCornerSeverityPct < 60) {
    return "MEDIUM";
  }
  if (visionCornerSeverityPct < 85) {
    return "SHARP";
  }
  return "VERY SHARP";
```

Integrated autonomous controller

```
}
float getVisionCornerSpeedCapRPM() {
  if (!visionIsTrusted()) {
    return VISION_FALLBACK_RPM;
  }
  float severity = constrain( (float)visionCornerSeverityPct / 100.0f, 0.0f, 1.0f );
  // F = m*v^2/R means the speed limit should fall
  // non-linearly as curvature demand rises.
  float speedFactor = 1.0f / sqrtf( 1.0f + CORNER_CURVATURE_GAIN * severity );
  float capRPM = NORMAL_BASE_RPM * speedFactor;
  return constrain( capRPM, VISION_MIN_BASE_RPM, NORMAL_BASE_RPM );
}
float getVisionBaseRPM() {
  if (!visionIsTrusted()) {
    return VISION_FALLBACK_RPM;
  }
  int safeSpeedPct = constrain( visionSpeedPct, VISION_MIN_SPEED_PCT, VISION_MAX_SPEED_PCT );
  float cameraRequestedRPM = NORMAL_BASE_RPM * ( (float)safeSpeedPct / 100.0f );
  float cornerSpeedCapRPM = getVisionCornerSpeedCapRPM();
  float safeRPM = min( cameraRequestedRPM, cornerSpeedCapRPM );
  return constrain( safeRPM, VISION_MIN_BASE_RPM, NORMAL_BASE_RPM );
}
bool parseVisionPacket( char *packet ) {
  int steer = 0;
  int speed = 0;
  int confidence = 0;
  int bridgeConfidence = 0;
```

Integrated autonomous controller

```
  int cornerSeverity = 0;
```

ORIGINAL LINES 1,123-1,161

```
  // Backward compatible packet formats:
  // V,steer,speed,confidence
  // V,steer,speed,confidence,bridgeConfidence
  // V,steer,speed,confidence,bridgeConfidence,cornerSeverity
  int parsed = sscanf( packet, "V,%d,%d,%d,%d,%d", &steer, &speed, &confidence, &bridgeConfidence, &cornerSeverity );
  if ( parsed != 3 && parsed != 4 && parsed != 5 ) {
    return false;
  }
  visionSteerPct = constrain( steer, -100, 100 );
  visionSpeedPct = constrain( speed, VISION_MIN_SPEED_PCT, VISION_MAX_SPEED_PCT );
  visionConfidencePct = constrain( confidence, 0, 100 );
  visionBridgeConfidencePct = constrain( parsed >= 4 ? bridgeConfidence : 0, 0, 100 );
  visionCornerSeverityPct = constrain( parsed >= 5 ? cornerSeverity : 0, 0, 100 );
  visionPacketValid = true;
  lastVisionPacketTime = millis();
  return true;
}
void getWokwiCornerTestCommand( int &steerPct, int &speedPct, int &cornerSeverityPct ) {
  steerPct = WOKWI_VISION_STEER_PCT;
  speedPct = WOKWI_VISION_SPEED_PCT;
  cornerSeverityPct = WOKWI_VISION_CORNER_SEVERITY_PCT;
  if (!WOKWI_AUTO_CORNER_SPEED_TEST) {
    return;
  }
  if (!wokwiCornerTestStarted) {
```

Integrated autonomous controller

```
    wokwiCornerTestStarted = true;
    wokwiCornerTestStartTime = millis();
  }
  unsigned long elapsed = millis() - wokwiCornerTestStartTime;
  // 0-3 s: STRAIGHT
  if (elapsed < CORNER_TEST_GENTLE_MS) {
    steerPct = 0;
    speedPct = 100;
    cornerSeverityPct = 0;
    return;
  }
  // 3-6 s: GENTLE RIGHT
  if (elapsed < CORNER_TEST_MEDIUM_MS) {
    steerPct = 20;
```

ORIGINAL LINES 1,162-1,200

```
    speedPct = 100;
    cornerSeverityPct = 25;
    return;
  }
  // 6-9 s: MEDIUM LEFT
  if (elapsed < CORNER_TEST_SHARP_MS) {
    steerPct = -40;
    speedPct = 100;
    cornerSeverityPct = 50;
    return;
  }
  // 9-12 s: SHARP RIGHT
```

Integrated autonomous controller

```
  if (elapsed < CORNER_TEST_VERY_SHARP_MS) {
    steerPct = 60;
    speedPct = 100;
    cornerSeverityPct = 75;
    return;
  }
  // 12-15 s: VERY SHARP LEFT
  if (elapsed < CORNER_TEST_END_MS) {
    steerPct = -75;
    speedPct = 100;
    cornerSeverityPct = 100;
    return;
  }
  // >15 s: STRAIGHT AGAIN
  steerPct = 0;
  speedPct = 100;
  cornerSeverityPct = 0;
}
void updateVisionSerial() {
  // ================================================
  // WOKWI INTERNAL VISION MODE
  // ================================================
  // This bypasses the custom Pi UART chip entirely.
  // It tests the SAME downstream STM32 navigation logic
  // that the real Pi packets will use later.
  // ================================================
  if (USE_WOKWI_INTERNAL_VISION_SIM) {
```

ORIGINAL LINES 1,201-1,239

Integrated autonomous controller

```
    unsigned long now = millis();
    if ( now - previousWokwiVisionTime < WOKWI_VISION_INTERVAL_MS ) {
      return;
    }
    previousWokwiVisionTime = now;
    if (!WOKWI_VISION_ONLINE) {
      // Send nothing. The normal 250 ms timeout will
      // naturally change VISION STATUS to OFFLINE / STALE.
      return;
    }
    int simulatedSteerPct = 0;
    int simulatedSpeedPct = 100;
    int simulatedCornerSeverityPct = 0;
    getWokwiCornerTestCommand( simulatedSteerPct, simulatedSpeedPct, simulatedCornerSeverityPct );
    visionSteerPct = constrain( simulatedSteerPct, -100, 100 );
    visionSpeedPct = constrain( simulatedSpeedPct, VISION_MIN_SPEED_PCT, VISION_MAX_SPEED_PCT );
    visionCornerSeverityPct = constrain( simulatedCornerSeverityPct, 0, 100 );
    visionConfidencePct = constrain( WOKWI_VISION_CONFIDENCE_PCT, 0, 100 );
    visionBridgeConfidencePct = constrain( wokwiSimulatedBridgeConfidencePct, 0, 100 );
    visionPacketValid = true;
    lastVisionPacketTime = now;
    // Keep the existing diagnostics useful.
    snprintf( piLastRawPacket, sizeof(piLastRawPacket), "SIM,V,%d,%d,%d,%d,%d", visionSteerPct, visionSpeedPct, visionConfidencePct,
      visionBridgeConfidencePct, visionCornerSeverityPct );
    piGoodPacketCount++;
    return;
  }
```

Integrated autonomous controller

```
  // ================================================
  // PHYSICAL RASPBERRY PI UART MODE
  // ================================================
  // Set USE_WOKWI_INTERNAL_VISION_SIM = false to use this
  // path. The real Pi sends:
  //   V,<steerPct>,<speedPct>,<confidencePct>,<bridgeConfidencePct>,<cornerSeverityPct>\n
  // Older 4-field and 5-field packets remain accepted.
  // ================================================
  while (PiSerial.available() > 0) {
    char c = (char)PiSerial.read();
    piRxByteCount++;
    if ( c == '\n' || c == '\r' ) {
```

ORIGINAL LINES 1,240-1,278

```
      if (visionRxIndex > 0) {
        visionRxBuffer[ visionRxIndex ] = '\0';
        strncpy( piLastRawPacket, visionRxBuffer, sizeof(piLastRawPacket) - 1 );
        piLastRawPacket[ sizeof(piLastRawPacket) - 1 ] = '\0';
        if ( parseVisionPacket( visionRxBuffer ) ) {
          piGoodPacketCount++;
        }
        else {
          piBadPacketCount++;
        }
        visionRxIndex = 0;
      }
      continue;
    }
```

Integrated autonomous controller

```
    if ( visionRxIndex < sizeof(visionRxBuffer) - 1 ) {
      visionRxBuffer[ visionRxIndex++ ] = c;
    }
    else {
      visionRxIndex = 0;
      piBadPacketCount++;
    }
  }
}
// ====================================================
// CAMERA PREDICTIVE MOTOR TARGETS
// ====================================================
void applyVisionGuidanceAtBase( float requestedBaseRPM ) {
  float baseRPM = min( getVisionBaseRPM(), requestedBaseRPM );
  float normalizedSteer = (float)visionSteerPct / 100.0f;
  float correctionRPM = VISION_MAX_STEER_RPM * normalizedSteer;
  // Positive steering means RIGHT:
  // left wheel faster, right wheel slower.
  leftTargetRPM = baseRPM + correctionRPM;
  rightTargetRPM = baseRPM - correctionRPM;
  leftTargetRPM = constrain( leftTargetRPM, MIN_TARGET_RPM, MAX_TARGET_RPM );
  rightTargetRPM = constrain( rightTargetRPM, MIN_TARGET_RPM, MAX_TARGET_RPM );
}
void applyVisionGuidance() {
  applyVisionGuidanceAtBase( NORMAL_BASE_RPM );
```

ORIGINAL LINES 1,279-1,317

```
}
```

Integrated autonomous controller

```
// ====================================================
// ROAD-BOUNDARY SENSORS
// ====================================================
void readBoundarySensors() {
  for (int i = 0; i < 8; i++) {
    sensor[i] = digitalRead(IR_PINS[i]);
  }
}
void analyseRoadBoundaries() {
  float weightedSum = 0.0f;
  activeBoundarySensors = 0;
  leftBoundarySeverity = 0.0f;
  rightBoundarySeverity = 0.0f;
  boundarySeverity = 0.0f;
  bool leftActive = false;
  bool rightActive = false;
  for (int i = 0; i < 8; i++) {
    if (sensor[i] != HIGH) continue;
    float weight = BOUNDARY_STEER_WEIGHTS[i];
    float severity = fabsf(weight);
    weightedSum += weight;
    activeBoundarySensors++;
    if (severity > boundarySeverity) boundarySeverity = severity;
    if (i <= 3) {
      leftActive = true;
      if (severity > leftBoundarySeverity) leftBoundarySeverity = severity;
    }
    else {
```

Integrated autonomous controller

```
      rightActive = true;
      if (severity > rightBoundarySeverity) rightBoundarySeverity = severity;
    }
  }
  boundaryDetected = activeBoundarySensors > 0;
  centreBoundaryDetected = ( sensor[3] == HIGH || sensor[4] == HIGH );
  bothSidesBoundaryDetected = leftActive && rightActive;
  if (!boundaryDetected) {
    boundarySteerError = 0.0f;
    return;
```

ORIGINAL LINES 1,318-1,356

```
  }
  boundarySteerError = weightedSum / (float)activeBoundarySensors;
  // Keep a useful directional memory. Do not replace it
  // with an almost perfectly balanced / ambiguous value.
  if ( fabsf(boundarySteerError) > 0.25f ) {
    lastBoundarySteerError = boundarySteerError;
  }
}
// ====================================================
// TOF READ
// ====================================================
bool readToFDistance( uint8_t address, uint16_t &distanceMM ) {
  Wire.beginTransmission(address);
  Wire.write( (uint8_t)( SIM_RANGE_REGISTER >> 8 ) );
  Wire.write( (uint8_t)( SIM_RANGE_REGISTER & 0xFF ) );
  // NORMAL STOP TRANSACTION
```

Integrated autonomous controller

```
  uint8_t result = Wire.endTransmission();
  if (result != 0) return false;
  uint8_t received = Wire.requestFrom( address, (uint8_t)2 );
  if ( received != 2 || Wire.available() < 2 ) {
    return false;
  }
  uint8_t highByte = Wire.read();
  uint8_t lowByte = Wire.read();
  distanceMM = ((uint16_t)highByte << 8) | lowByte;
  // ================================================
  // VL53L1X SENSOR-FAULT DETECTION
  // The Wokwi fault simulator returns 0xFFFF (65535)
  // when Sensor Fault = 1. A zero reading is also
  // treated as invalid.
  // ================================================
  if ( distanceMM == 0 || distanceMM >= 5000 ) {
    return false;
  }
  return true;
}
// ================================================
// TOF + TTC UPDATE
// ================================================
```

ORIGINAL LINES 1,357-1,395

```
void updateToF() {
  unsigned long now = millis();
  if ( now - previousToFTime < TOF_INTERVAL_MS ) {
```

Integrated autonomous controller

```
1360      return;
1361    }
1362    float dt = (now - previousToFTime) / 1000.0f;
1363    previousToFTime = now;
1364    uint16_t newLeft = 0;
1365    uint16_t newRight = 0;
1366    bool leftOK = false;
1367    bool rightOK = false;
1368    if (USE_WOKWI_INTERNAL_TOF_SIM) {
1369      // Internal ToF mode is retained only as a fallback.
1370      // The current Wokwi race build keeps this FALSE so the
1371      // custom VL53L1X distance sliders are used.
1372      newLeft = 1500;
1373      newRight = 1500;
1374      leftOK = true;
1375      rightOK = true;
1376    }
1377    else {
1378      // Interactive Wokwi mode:
1379      // read the actual custom VL53L1X chips.
1380      // Moving either distance slider changes these values.
1381      leftOK = readToFDistance( LEFT_TOF_ADDR, newLeft );
1382      rightOK = readToFDistance( RIGHT_TOF_ADDR, newRight );
1383    }
1384    if (!leftOK || !rightOK) {
1385      tofValid = false;
1386      previousToFValid = false;
1387      return;
```

Integrated autonomous controller

```
  }
  tofValid = true;
  leftDistance = newLeft;
  rightDistance = newRight;
  if ( previousToFValid && dt > 0.001f ) {
    leftClosingSpeed = ( (float)previousLeftDistance - (float)leftDistance ) / dt;
    rightClosingSpeed = ( (float)previousRightDistance - (float)rightDistance ) / dt;
    if ( leftClosingSpeed > MIN_CLOSING_SPEED_MM_S ) {
```

ORIGINAL LINES 1,396-1,434

```
      leftTTC = (float)leftDistance / leftClosingSpeed;
    }
    else {
      leftTTC = -1.0f;
    }
    if ( rightClosingSpeed > MIN_CLOSING_SPEED_MM_S ) {
      rightTTC = (float)rightDistance / rightClosingSpeed;
    }
    else {
      rightTTC = -1.0f;
    }
  }
  else {
    leftClosingSpeed = 0.0f;
    rightClosingSpeed = 0.0f;
    leftTTC = -1.0f;
    rightTTC = -1.0f;
  }
```

Integrated autonomous controller

```
  previousLeftDistance = leftDistance;
  previousRightDistance = rightDistance;
  previousToFValid = true;
}
// ====================================================
// INA219 REGISTER READ
// Uses WRITE -> STOP -> READ -> STOP.
// This matches the stable transaction style already used
// by the shared Wokwi I2C simulation.
// ====================================================
bool readINA219Register( uint8_t reg, uint16_t &value ) {
  Wire.beginTransmission( INA219_ADDR );
  Wire.write(reg);
  if ( Wire.endTransmission() != 0 ) {
    return false;
  }
  uint8_t received = Wire.requestFrom( INA219_ADDR, (uint8_t)2 );
  if ( received != 2 || Wire.available() < 2 ) {
    return false;
  }
  uint8_t highByte = Wire.read();
```

ORIGINAL LINES 1,435-1,473

```
  uint8_t lowByte = Wire.read();
  value = ((uint16_t)highByte << 8) | lowByte;
  return true;
}
// ====================================================
```

Integrated autonomous controller

```
// INA219 UPDATE
// ================================================
void updateINA219( bool forceRead = false ) {
  unsigned long now = millis();
  if ( !forceRead && now - previousINATime < INA_INTERVAL_MS ) {
    return;
  }
  previousINATime = now;
  if (USE_WOKWI_INTERNAL_INA_SIM) {
    inaValid = WOKWI_INA_ONLINE;
    if (!inaValid) {
      motorStallDetected = false;
      stallConditionStart = 0;
      return;
    }
    busVoltageV = WOKWI_BUS_VOLTAGE_V;
    motorCurrentMA = WOKWI_MOTOR_CURRENT_MA;
    return;
  }
  uint16_t rawBusVoltage = 0;
  uint16_t rawCurrent = 0;
  bool busOK = readINA219Register( INA219_REG_BUS_VOLTAGE, rawBusVoltage );
  bool currentOK = readINA219Register( INA219_REG_CURRENT, rawCurrent );
  if (!busOK || !currentOK) {
    inaValid = false;
    motorStallDetected = false;
    stallConditionStart = 0;
    return;
```

Integrated autonomous controller

```
    }
    inaValid = true;
    // INA219-style bus voltage register:
    // voltage bits are [15:3], 4 mV per bit.
    uint16_t busVoltageMV = (rawBusVoltage >> 3) * 4U;
    busVoltageV = (float)busVoltageMV / 1000.0f;
```

ORIGINAL LINES 1,474-1,512

```
    // Wokwi behavioural-model convention:
    // 1 raw current count = 1 mA.
    motorCurrentMA = (float)( (int16_t)rawCurrent );
}
// ======================================================
// MOTOR STALL DETECTION
// ======================================================
// A stall is only declared when ALL are true:
// 1. INA219 current is high.
// 2. The controller is commanding meaningful motion.
// 3. Actual wheel speed is very low.
// 4. The condition persists for STALL_CONFIRM_MS.
// The time confirmation prevents startup or brief
// transients from immediately causing a false stall.
// ======================================================
void updateMotorStallDetection() {
    // ======================================================
    // HARD STALL LATCH
    // ======================================================
    // Once a real stall is confirmed, current falling back
```

Integrated autonomous controller

```
  // to normal MUST NOT automatically restart the robot.
  // The robot stays stopped until the operator deliberately
  // cycles the MAIN switch:
  //   MAIN ON -> stall detected -> latched stop
  //   MAIN OFF -> latch is cleared while motors are disabled
  //   MAIN ON  -> autonomous control may start again
  // This is safer for a fully autonomous racing robot because
  // it prevents repeated automatic attempts against a jammed
  // wheel or trapped drivetrain.
  // ================================================
  if (motorStallLatched) {
    motorStallDetected = true;
    return;
  }
  if (!inaValid) {
    motorStallDetected = false;
    stallCandidateMask = 0;
    stallLatchedSideMask = 0;
    stallConditionStart = 0;
```

ORIGINAL LINES 1,513-1,551

```
    return;
  }
  // Do not diagnose a mechanical stall from corrupt
  // encoder samples.
  if (!leftRPMValid || !rightRPMValid) {
    motorStallDetected = false;
    stallCandidateMask = 0;
```

Integrated autonomous controller

```
    stallConditionStart = 0;
    return;
  }
  // ====================================================
  // PER-WHEEL STALL CHECK
  // ====================================================
  // Previous logic averaged the two wheel RPM values.
  // That misses a one-wheel stall:
  //   left = 0 RPM
  //   right = 38 RPM
  //   average = 19 RPM
  // 19 RPM is above the 5 RPM stall threshold, so the
  // old controller incorrectly concluded "NO STALL".
  // We now evaluate each wheel independently.
  // ====================================================
  bool highCurrent = motorCurrentMA >= STALL_CURRENT_MA;
  bool leftCommanded = fabsf( leftRampedTargetRPM ) > STALL_COMMAND_RPM_THRESHOLD;
  bool rightCommanded = fabsf( rightRampedTargetRPM ) > STALL_COMMAND_RPM_THRESHOLD;
  bool leftTooSlow = fabsf( leftRPM ) < STALL_RPM_THRESHOLD;
  bool rightTooSlow = fabsf( rightRPM ) < STALL_RPM_THRESHOLD;
  bool leftStallCondition = highCurrent && leftCommanded && leftTooSlow;
  bool rightStallCondition = highCurrent && rightCommanded && rightTooSlow;
  stallCandidateMask = 0;
  if (leftStallCondition) {
    stallCandidateMask |= 0x01;
  }
  if (rightStallCondition) {
    stallCandidateMask |= 0x02;
```

Integrated autonomous controller

```
  }
  bool stallCondition = stallCandidateMask != 0;
  unsigned long now = millis();
  if (!stallCondition) {
```

ORIGINAL LINES 1,552-1,590

```
    motorStallDetected = false;
    stallConditionStart = 0;
    return;
  }
  if (stallConditionStart == 0) {
    stallConditionStart = now;
    motorStallDetected = false;
    return;
  }
  if ( now - stallConditionStart >= STALL_CONFIRM_MS ) {
    motorStallLatched = true;
    motorStallDetected = true;
    stallLatchedSideMask = stallCandidateMask;
  }
  else {
    motorStallDetected = false;
  }
}
void resetMotorStallLatchWhileSystemOff() {
  // This function must only be called while MAIN SWITCH = OFF.
  // The motor driver is already disabled in SYSTEM OFF, so it
  // is safe to clear the software latch here. The robot cannot
```

Integrated autonomous controller

```
  // move again until the operator deliberately switches MAIN ON.
  motorStallLatched = false;
  motorStallDetected = false;
  stallCandidateMask = 0;
  stallLatchedSideMask = 0;
  stallConditionStart = 0;
}
const char *stallSideName() {
  uint8_t mask = motorStallLatched ? stallLatchedSideMask : stallCandidateMask;
  if (mask == 0x03) {
    return "BOTH";
  }
  if (mask == 0x01) {
    return "LEFT";
  }
  if (mask == 0x02) {
    return "RIGHT";
```

ORIGINAL LINES 1,591-1,629

```
  }
  return "NONE";
}
// ================================================
// CURRENT STATUS TEXT
// ================================================
const char *currentStatusName() {
  if (!inaValid) return "SENSOR ERROR";
  if (motorStallDetected) return "STALL";
```

Integrated autonomous controller

```
  if ( motorCurrentMA >= HIGH_CURRENT_MA ) {
    return "HIGH";
  }
  return "NORMAL";
}
// ======================================================
// MPU WRITE
// ======================================================
bool writeMPURegister( uint8_t reg, uint8_t value ) {
  Wire.beginTransmission( MPU_ADDR );
  Wire.write(reg);
  Wire.write(value);
  return Wire.endTransmission() == 0;
}
// ======================================================
// MPU READ BYTE
// IMPORTANT:
// NO REPEATED START.
// We deliberately use:
// WRITE -> STOP
// READ  -> STOP
// because this is stable with the current Wokwi
// shared I2C simulation.
// ======================================================
bool readMPUByte( uint8_t reg, uint8_t &value ) {
  Wire.beginTransmission( MPU_ADDR );
  Wire.write(reg);
  if ( Wire.endTransmission() != 0 ) {
```

Integrated autonomous controller

```
    return false;
  }
```

ORIGINAL LINES 1,630-1,668

```
  uint8_t received = Wire.requestFrom( MPU_ADDR, (uint8_t)1 );
  if ( received != 1 || Wire.available() < 1 ) {
    return false;
  }
  value = Wire.read();
  return true;
}
// ======================================================
// MPU INITIALISATION
// ======================================================
bool initialiseMPU() {
  if (USE_WOKWI_INTERNAL_IMU_SIM) {
    gyroZ = WOKWI_GYRO_Z_DPS;
    return WOKWI_IMU_ONLINE;
  }
  if ( !writeMPURegister( MPU_PWR_MGMT_1, 0x00 ) ) {
    return false;
  }
  if ( !writeMPURegister( MPU_GYRO_CONFIG, 0x00 ) ) {
    return false;
  }
  uint8_t who = 0;
  if ( !readMPUByte( MPU_WHO_AM_I, who ) ) {
    return false;
```

Integrated autonomous controller

```
  }
  return who == 0x68;
}
// =======================================================
// READ GYRO Z
// ALSO USES NORMAL STOP TRANSACTIONS.
// =======================================================
bool readGyroZ() {
  if (USE_WOKWI_INTERNAL_IMU_SIM) {
    gyroZ = WOKWI_GYRO_Z_DPS;
    pitchDeg = wokwiSimulatedPitchDeg;
    return WOKWI_IMU_ONLINE;
  }
  // Physical MPU6050:
  // read accel XYZ + temp + gyro XYZ in one 14-byte burst.
```

ORIGINAL LINES 1,669-1,707

```
  Wire.beginTransmission( MPU_ADDR );
  Wire.write( MPU_ACCEL_XOUT_H );
  if ( Wire.endTransmission() != 0 ) {
    return false;
  }
  uint8_t received = Wire.requestFrom( MPU_ADDR, (uint8_t)14 );
  if ( received != 14 || Wire.available() < 14 ) {
    return false;
  }
  int16_t rawAx = ((int16_t)Wire.read() << 8) | Wire.read();
  int16_t rawAy = ((int16_t)Wire.read() << 8) | Wire.read();
```

Integrated autonomous controller

```
1680    int16_t rawAz = ((int16_t)Wire.read() << 8) | Wire.read();
1681    // Temperature (unused).
1682    Wire.read();
1683    Wire.read();
1684    // Gyro X and Y (unused here).
1685    Wire.read();
1686    Wire.read();
1687    Wire.read();
1688    Wire.read();
1689    int16_t rawGz = ((int16_t)Wire.read() << 8) | Wire.read();
1690    gyroZ = (float)rawGz / GYRO_SENSITIVITY;
1691    // Assumed physical orientation:
1692    // X points forward, Z points upward.
1693    // Change axis/sign after real mounting calibration if needed.
1694    float ax = (float)rawAx;
1695    float ay = (float)rawAy;
1696    float az = (float)rawAz;
1697    float measuredPitch = atan2f( -ax, sqrtf( ay * ay + az * az ) ) * 180.0f / PI;
1698    // Low-pass pitch for vibration/noise.
1699    pitchDeg = 0.80f * pitchDeg + 0.20f * measuredPitch;
1700    return true;
1701  }
1702  // ================================================
1703  // TTC HELPERS
1704  // ================================================
1705  float getMinimumTTC() {
1706    bool leftValid = leftTTC > 0.0f;
```

Integrated autonomous controller

```
1707    bool rightValid = rightTTC > 0.0f;
```

ORIGINAL LINES 1,708-1,746

```
1708    if (leftValid && rightValid) {
1709      return leftTTC < rightTTC ? leftTTC :
1710        rightTTC;
1711    }
1712    if (leftValid) return leftTTC;
1713    if (rightValid) return rightTTC;
1714    return 999.0f;
1715  }
1716  // ====================================================
1717  // AVOIDANCE DIRECTION
1718  // -1 = LEFT
1719  // +1 = RIGHT
1720  //  0 = UNDETERMINED
1721  // ====================================================
1722  int chooseAvoidanceDirection() {
1723    int direction = 0;
1724    // First choose the safer side using TTC when available.
1725    if ( leftTTC > 0.0f && rightTTC > 0.0f ) {
1726      if ( fabsf( leftTTC - rightTTC ) > 0.15f ) {
1727        if (leftTTC < rightTTC) direction = +1; // obstacle closes sooner on left -> go right
1728        else direction = -1; // obstacle closes sooner on right -> go left
1729      }
1730    }
1731    else if (leftTTC > 0.0f) {
1732      direction = +1;
```

Integrated autonomous controller

```
  }
  else if (rightTTC > 0.0f) {
    direction = -1;
  }
  // If TTC did not decide, use the measured distances.
  if (direction == 0) {
    int difference = (int)leftDistance - (int)rightDistance;
    if ( abs(difference) > SIDE_TOLERANCE_MM ) {
      if (leftDistance < rightDistance) direction = +1;
      else direction = -1;
    }
  }
  // ================================================
  // ROAD-BOUNDARY VETO
```

ORIGINAL LINES 1,747-1,785

```
  // ================================================
  // Do not avoid an obstacle by steering directly into
  // a nearby road boundary.
  // direction -1 = LEFT
  // direction +1 = RIGHT
  // Severity >= 2 means the boundary has reached beyond
  // the far-outside sensor and we treat that side as
  // unsafe for an obstacle-avoidance turn.
  // ================================================
  if ( direction < 0 && leftBoundarySeverity >= 2.0f ) {
    return 0;
  }
```

Integrated autonomous controller

```
  if ( direction > 0 && rightBoundarySeverity >= 2.0f ) {
    return 0;
  }
  return direction;
}
// ======================================================
// SKID DETECTION
// ======================================================
void updateSkidDetection() {
  if (!leftRPMValid || !rightRPMValid) {
    possibleSkid = false;
    return;
  }
  float rpmDifference = fabsf( leftRPM - rightRPM );
  float averageRPM = ( fabsf(leftRPM) + fabsf(rightRPM) ) / 2.0f;
  possibleSkid = ( averageRPM > 10.0f && rpmDifference < 3.0f && fabsf(gyroZ) > 25.0f );
}
// ======================================================
// AUTONOMOUS BRIDGE PHASE ESTIMATION
// ======================================================
const char *bridgePhaseName() {
  switch (bridgePhase) {
    case BRIDGE_NONE:     return "NONE";
    case BRIDGE_APPROACH: return "APPROACH";
    case BRIDGE_CLIMB:    return "CLIMB";
    case BRIDGE_CREST:    return "CREST";
    case BRIDGE_DESCENT:  return "DESCENT";
```

ORIGINAL LINES 1,786-1,824

Integrated autonomous controller

```
    case BRIDGE_EXIT:     return "EXIT";
  }
  return "INVALID";
}
void updateBridgePhase() {
  if (pitchDeg > maximumBridgeClimbPitchDeg) {
    maximumBridgeClimbPitchDeg = pitchDeg;
  }
  switch (bridgePhase) {
    case BRIDGE_NONE:
      maximumBridgeClimbPitchDeg = 0.0f;
      bridgeExitStartTime = 0;
      if ( visionIsTrusted() && visionBridgeConfidencePct >= BRIDGE_CAMERA_ENTER_CONFIDENCE_PCT ) {
        bridgePhase = BRIDGE_APPROACH;
        return;
      }
      // IMU fallback if camera prediction is missed.
      if ( imuValid && pitchDeg >= BRIDGE_CLIMB_ENTER_PITCH_DEG ) {
        bridgePhase = BRIDGE_CLIMB;
        maximumBridgeClimbPitchDeg = pitchDeg;
        return;
      }
      if ( imuValid && pitchDeg <= BRIDGE_DESCENT_ENTER_PITCH_DEG ) {
        bridgePhase = BRIDGE_DESCENT;
        return;
      }
      break;
```

Integrated autonomous controller

```
    case BRIDGE_APPROACH:
      if ( imuValid && pitchDeg >= BRIDGE_CLIMB_ENTER_PITCH_DEG ) {
        bridgePhase = BRIDGE_CLIMB;
        maximumBridgeClimbPitchDeg = pitchDeg;
        return;
      }
      // Cancel false camera bridge detection.
      if ( visionBridgeConfidencePct < BRIDGE_CAMERA_CLEAR_CONFIDENCE_PCT && fabsf(pitchDeg) < BRIDGE_FLAT_PITCH_DEG ) {
        bridgePhase = BRIDGE_NONE;
        return;
      }
      break;
```

ORIGINAL LINES 1,825-1,863

```
    case BRIDGE_CLIMB:
      if ( pitchDeg <= BRIDGE_DESCENT_ENTER_PITCH_DEG ) {
        bridgePhase = BRIDGE_DESCENT;
        return;
      }
      if ( maximumBridgeClimbPitchDeg >= BRIDGE_CLIMB_ENTER_PITCH_DEG && fabsf(pitchDeg) <= BRIDGE_CREST_PITCH_DEG ) {
        bridgePhase = BRIDGE_CREST;
        return;
      }
      break;
    case BRIDGE_CREST:
      if ( pitchDeg <= BRIDGE_DESCENT_ENTER_PITCH_DEG ) {
        bridgePhase = BRIDGE_DESCENT;
        return;
```

Integrated autonomous controller

```
      }
      if ( pitchDeg >= BRIDGE_CLIMB_ENTER_PITCH_DEG ) {
        bridgePhase = BRIDGE_CLIMB;
        return;
      }
      break;
    case BRIDGE_DESCENT:
      if ( fabsf(pitchDeg) <= BRIDGE_FLAT_PITCH_DEG ) {
        bridgePhase = BRIDGE_EXIT;
        bridgeExitStartTime = millis();
        return;
      }
      break;
    case BRIDGE_EXIT:
      if ( pitchDeg <= BRIDGE_DESCENT_ENTER_PITCH_DEG ) {
        bridgePhase = BRIDGE_DESCENT;
        bridgeExitStartTime = 0;
        return;
      }
      if ( fabsf(pitchDeg) <= BRIDGE_FLAT_PITCH_DEG && visionBridgeConfidencePct < BRIDGE_CAMERA_CLEAR_CONFIDENCE_PCT ) {
        if ( millis() - bridgeExitStartTime >= BRIDGE_EXIT_CONFIRM_MS ) {
          bridgePhase = BRIDGE_NONE;
          maximumBridgeClimbPitchDeg = 0.0f;
          bridgeExitStartTime = 0;
          return;
```

ORIGINAL LINES 1,864-1,900

```
        }
```

Integrated autonomous controller

```
      }
      else {
        bridgeExitStartTime = millis();
      }
      break;
  }
}
// ====================================================
// BRIDGE-SAFE CAMERA STEERING
// ====================================================
void applyBridgeGuidance( float baseRPM, float maximumSteerRPM ) {
  if (!visionIsTrusted()) {
    applyRoadKeeping( baseRPM );
    return;
  }
  float safeBridgeBaseRPM = min( baseRPM, getVisionCornerSpeedCapRPM() );
  float normalizedSteer = constrain( (float)visionSteerPct / 100.0f, -1.0f, 1.0f );
  float correctionRPM = maximumSteerRPM * normalizedSteer;
  leftTargetRPM = constrain( safeBridgeBaseRPM + correctionRPM, MIN_TARGET_RPM, min( MAX_TARGET_RPM, safeBridgeBaseRPM + maximumSteerRPM ) );
  rightTargetRPM = constrain( safeBridgeBaseRPM - correctionRPM, MIN_TARGET_RPM, min( MAX_TARGET_RPM, safeBridgeBaseRPM + maximumSteerRPM ) );
}
// ====================================================
// STATE DECISION
// ====================================================
void determineNavigationState() {
  // ====================================================
```

Integrated autonomous controller

```
  // 1. ABSOLUTE SAFETY: E-STOP
  // ================================================
  if (emergencyStopActive) {
    currentState = STATE_EMERGENCY_STOP;
    return;
  }
  // ================================================
  // 2. MAIN SWITCH OFF
  // ================================================
  if (!mainSwitchOn) {
```

ORIGINAL LINES 1,901-1,939

```
    currentState = STATE_SYSTEM_OFF;
    return;
  }
  // ================================================
  // 3. MOTOR / ELECTRICAL PROTECTION
  // ================================================
  if (motorStallDetected) {
    currentState = STATE_MOTOR_STALL;
    return;
  }
  // ================================================
  // 3. COLLISION SAFETY: ToF + TTC
  // ================================================
  // Other race cars and track walls are dynamic hazards.
  // TTC makes the decision depend on closing rate as well
  // as raw distance.
```

Integrated autonomous controller

```
  // ================================================
  if (tofValid) {
    uint16_t nearestDistance = leftDistance < rightDistance ? leftDistance :
      rightDistance;
    float minimumTTC = getMinimumTTC();
    int avoidDirection = chooseAvoidanceDirection();
    // Critical collision risk
    if ( nearestDistance <= CRITICAL_RANGE_MM || minimumTTC <= TTC_CRITICAL_S ) {
      if (avoidDirection > 0) currentState = STATE_CRITICAL_AVOID_RIGHT;
      else if (avoidDirection < 0) currentState = STATE_CRITICAL_AVOID_LEFT;
      else currentState = STATE_CRITICAL_STOP;
      return;
    }
    // Active avoidance
    if ( nearestDistance <= AVOID_RANGE_MM || minimumTTC <= TTC_AVOID_S ) {
      if (avoidDirection > 0) currentState = STATE_AVOID_RIGHT;
      else if (avoidDirection < 0) currentState = STATE_AVOID_LEFT;
      else currentState = STATE_NO_SAFE_PATH;
      return;
    }
    // Early caution
    if ( nearestDistance <= CAUTION_RANGE_MM || minimumTTC <= TTC_CAUTION_S ) {
      currentState = STATE_CAUTION;
```

ORIGINAL LINES 1,940-1,978

```
      return;
    }
  }
```

Integrated autonomous controller

```
1943    // ================================================
1944    // 4. SKID / STABILITY PROTECTION
1945    // ================================================
1946    if (possibleSkid) {
1947      currentState = STATE_SKID_CAUTION;
1948      return;
1949    }
1950    // ================================================
1951    // 5. IR ROAD-BOUNDARY OVERRIDE
1952    // ================================================
1953    // Camera predicts what is coming.
1954    // IR protects what is happening under the vehicle NOW.
1955    // ================================================
1956    if (boundaryDetected) {
1957      if ( centreBoundaryDetected && fabsf(boundarySteerError) <= 0.25f ) {
1958        currentState = STATE_BOUNDARY_ESCAPE;
1959        return;
1960      }
1961      if (boundarySteerError > 0.25f) {
1962        currentState = STATE_BOUNDARY_CORRECT_RIGHT;
1963        return;
1964      }
1965      if (boundarySteerError < -0.25f) {
1966        currentState = STATE_BOUNDARY_CORRECT_LEFT;
1967        return;
1968      }
1969      currentState = STATE_BOUNDARY_CAUTION;
1970      return;
```

Integrated autonomous controller

```
1971    }
1972    // ================================================
1973    // 6. ToF FAULT -> DEGRADED SPEED
1974    // ================================================
1975    // Camera + IR can still navigate, but full racing speed
1976    // is not allowed without the car/wall collision layer.
1977    // ================================================
1978    if (!tofValid) {
```

ORIGINAL LINES 1,979-2,017

```
1979      currentState = STATE_SENSOR_FAULT;
1980      return;
1981    }
1982    // ================================================
1983    // 8. AUTONOMOUS BRIDGE HANDLING
1984    // ================================================
1985    switch (bridgePhase) {
1986      case BRIDGE_APPROACH:
1987        currentState = STATE_BRIDGE_APPROACH;
1988        return;
1989      case BRIDGE_CLIMB:
1990        currentState = STATE_BRIDGE_CLIMB;
1991        return;
1992      case BRIDGE_CREST:
1993        currentState = STATE_BRIDGE_CREST;
1994        return;
1995      case BRIDGE_DESCENT:
1996        currentState = STATE_BRIDGE_DESCENT;
```

Integrated autonomous controller

```
      return;
    case BRIDGE_EXIT:
      currentState = STATE_BRIDGE_EXIT;
      return;
    case BRIDGE_NONE:
      break;
  }
  // ================================================
  // 9. CAMERA PREDICTIVE LOOKAHEAD
  // ================================================
  if (visionIsTrusted()) {
    currentState = STATE_VISION_DRIVE;
    return;
  }
  // Camera missing / stale / low confidence.
  currentState = STATE_VISION_FALLBACK;
}
// ================================================
// CHOOSE BOUNDARY ESCAPE DIRECTION
// ================================================
// Return:
```

ORIGINAL LINES 2,018-2,056

```
//   -1 = turn LEFT
//   +1 = turn RIGHT
// This is mainly for an ambiguous centre-boundary hit
// such as 00011000. We first use directional memory,
// then use ToF free space, then use a deterministic
```

Integrated autonomous controller

```
// right-turn fallback for the Wokwi test.
// ====================================================
int chooseBoundaryEscapeDirection() {
  if (lastBoundarySteerError > 0.25f) return +1;
  if (lastBoundarySteerError < -0.25f) return -1;
  if (tofValid) {
    if ( leftDistance > rightDistance + SIDE_TOLERANCE_MM ) {
      return -1;
    }
    if ( rightDistance > leftDistance + SIDE_TOLERANCE_MM ) {
      return +1;
    }
  }
  return +1;
}
// ====================================================
// ROAD KEEPING / BOUNDARY AVOIDANCE
// ====================================================
void applyRoadKeeping( float requestedBaseRPM ) {
  // Clear road:
  // all IR sensors are over the drivable road surface.
  if (!boundaryDetected) {
    leftTargetRPM = requestedBaseRPM;
    rightTargetRPM = requestedBaseRPM;
    return;
  }
  // Reduce speed as the boundary moves closer to the
  // centre of the sensor array.
```

Integrated autonomous controller

```
  float baseRPM = requestedBaseRPM;
  if (boundarySeverity >= 3.5f) {
    baseRPM = min( baseRPM, BOUNDARY_CENTRE_RPM );
  }
  else if (boundarySeverity >= 2.5f) {
    baseRPM = min( baseRPM, BOUNDARY_INNER_RPM );
```

ORIGINAL LINES 2,057-2,095

```
  }
  else if (bothSidesBoundaryDetected) {
    baseRPM = min( baseRPM, BOUNDARY_CAUTION_RPM );
  }
  float correction = K_BOUNDARY_STEER * boundarySteerError;
  // Positive correction means the boundary is on the
  // LEFT, therefore the left wheel speeds up and the
  // right wheel slows down to turn the robot RIGHT.
  leftTargetRPM = baseRPM + correction;
  rightTargetRPM = baseRPM - correction;
  leftTargetRPM = constrain( leftTargetRPM, MIN_TARGET_RPM, MAX_TARGET_RPM );
  rightTargetRPM = constrain( rightTargetRPM, MIN_TARGET_RPM, MAX_TARGET_RPM );
}
// ======================================================
// BOUNDARY ESCAPE
// ======================================================
void applyBoundaryEscape() {
  int direction = chooseBoundaryEscapeDirection();
  if (direction > 0) {
    // Turn RIGHT strongly.
```

Integrated autonomous controller

```
    leftTargetRPM = AVOID_FAST_RPM;
    rightTargetRPM = CRITICAL_AVOID_SLOW_RPM;
  }
  else {
    // Turn LEFT strongly.
    leftTargetRPM = CRITICAL_AVOID_SLOW_RPM;
    rightTargetRPM = AVOID_FAST_RPM;
  }
}
// ====================================================
// APPLY NAVIGATION TARGETS
// ====================================================
void applyNavigationTargets() {
  switch (currentState) {
    case STATE_EMERGENCY_STOP:
    case STATE_SYSTEM_OFF:
      leftTargetRPM = 0.0f;
      rightTargetRPM = 0.0f;
      break;
```

ORIGINAL LINES 2,096-2,134

```
    case STATE_VISION_DRIVE:
      applyVisionGuidance();
      break;
    case STATE_VISION_FALLBACK:
      applyRoadKeeping( VISION_FALLBACK_RPM );
      break;
    case STATE_ROAD_DRIVE:
```

Integrated autonomous controller

```
2103        applyRoadKeeping( getVisionBaseRPM() );
2104        break;
2105      case STATE_BOUNDARY_CORRECT_LEFT:
2106        applyRoadKeeping( getVisionBaseRPM() );
2107        break;
2108      case STATE_BOUNDARY_CORRECT_RIGHT:
2109        applyRoadKeeping( getVisionBaseRPM() );
2110        break;
2111      case STATE_BOUNDARY_CAUTION:
2112        applyRoadKeeping( min( getVisionBaseRPM(), BOUNDARY_CAUTION_RPM ) );
2113        break;
2114      case STATE_BOUNDARY_ESCAPE:
2115        applyBoundaryEscape();
2116        break;
2117      case STATE_CAUTION:
2118        // Preserve road/camera guidance while reducing speed.
2119        if (boundaryDetected) {
2120          applyRoadKeeping( CAUTION_BASE_RPM );
2121        }
2122        else if (visionIsTrusted()) {
2123          applyVisionGuidanceAtBase( CAUTION_BASE_RPM );
2124        }
2125        else {
2126          leftTargetRPM = CAUTION_BASE_RPM;
2127          rightTargetRPM = CAUTION_BASE_RPM;
2128        }
2129        break;
2130      case STATE_AVOID_LEFT:
```

Integrated autonomous controller

```
      leftTargetRPM = AVOID_SLOW_RPM;
      rightTargetRPM = AVOID_FAST_RPM;
      break;
    case STATE_AVOID_RIGHT:
```

ORIGINAL LINES 2,135-2,173

```
      leftTargetRPM = AVOID_FAST_RPM;
      rightTargetRPM = AVOID_SLOW_RPM;
      break;
    case STATE_CRITICAL_AVOID_LEFT:
      leftTargetRPM = CRITICAL_AVOID_SLOW_RPM;
      rightTargetRPM = CRITICAL_AVOID_FAST_RPM;
      break;
    case STATE_CRITICAL_AVOID_RIGHT:
      leftTargetRPM = CRITICAL_AVOID_FAST_RPM;
      rightTargetRPM = CRITICAL_AVOID_SLOW_RPM;
      break;
    case STATE_CRITICAL_STOP:
      leftTargetRPM = 0.0f;
      rightTargetRPM = 0.0f;
      break;
    case STATE_NO_SAFE_PATH:
      leftTargetRPM = 0.0f;
      rightTargetRPM = 0.0f;
      break;
    case STATE_MOTOR_STALL:
      leftTargetRPM = 0.0f;
      rightTargetRPM = 0.0f;
```

Integrated autonomous controller

```
      break;
    case STATE_SKID_CAUTION:
      if (boundaryDetected) {
        applyRoadKeeping( SKID_BASE_RPM );
      }
      else if (visionIsTrusted()) {
        applyVisionGuidanceAtBase( SKID_BASE_RPM );
      }
      else {
        leftTargetRPM = SKID_BASE_RPM;
        rightTargetRPM = SKID_BASE_RPM;
      }
      break;
    case STATE_BRIDGE_APPROACH:
      applyBridgeGuidance( BRIDGE_APPROACH_RPM, BRIDGE_APPROACH_STEER_RPM );
      break;
    case STATE_BRIDGE_CLIMB:
```

ORIGINAL LINES 2,174-2,212

```
      applyBridgeGuidance( BRIDGE_CLIMB_RPM, BRIDGE_CLIMB_STEER_RPM );
      break;
    case STATE_BRIDGE_CREST:
      applyBridgeGuidance( BRIDGE_CREST_RPM, BRIDGE_CREST_STEER_RPM );
      break;
    case STATE_BRIDGE_DESCENT:
      applyBridgeGuidance( BRIDGE_DESCENT_RPM, BRIDGE_DESCENT_STEER_RPM );
      break;
    case STATE_BRIDGE_EXIT:
```

Integrated autonomous controller

```
      applyBridgeGuidance( BRIDGE_EXIT_RPM, BRIDGE_EXIT_STEER_RPM );
      break;
    case STATE_SENSOR_FAULT:
      if (boundaryDetected) {
        applyRoadKeeping( SENSOR_FAULT_RPM );
      }
      else if (visionIsTrusted()) {
        applyVisionGuidanceAtBase( SENSOR_FAULT_RPM );
      }
      else {
        leftTargetRPM = SENSOR_FAULT_RPM;
        rightTargetRPM = SENSOR_FAULT_RPM;
      }
      break;
    default:
      leftTargetRPM = 0.0f;
      rightTargetRPM = 0.0f;
      break;
  }
}
// ====================================================
// STATE CHANGE HANDLING
// ====================================================
void handleStateChange() {
  if ( currentState == previousState ) {
    return;
  }
  leftIntegral = 0.0f;
```

Integrated autonomous controller

```
  rightIntegral = 0.0f;
  leftPreviousError = 0.0f;
```

ORIGINAL LINES 2,213-2,251

```
  rightPreviousError = 0.0f;
  previousState = currentState;
}
// ====================================================
// IMMEDIATE HARD-STOP OUTPUT
// ====================================================
bool isHardStopState() {
  return currentState == STATE_EMERGENCY_STOP || currentState == STATE_SYSTEM_OFF || currentState == STATE_CRITICAL_STOP ||
    currentState == STATE_NO_SAFE_PATH || currentState == STATE_MOTOR_STALL;
}
void applyHardStopOutputs() {
  leftTargetRPM = 0.0f;
  rightTargetRPM = 0.0f;
  leftRampedTargetRPM = 0.0f;
  rightRampedTargetRPM = 0.0f;
  leftPWM = 0;
  rightPWM = 0;
  leftIntegral = 0.0f;
  rightIntegral = 0.0f;
  digitalWrite(PWMA, LOW);
  digitalWrite(PWMB, LOW);
}
// ====================================================
// FAST RACE / SAFETY CONTROL
```

Integrated autonomous controller

```
// ======================================================
// This is the fast steering loop.
// It reads ONLY the IR boundary array and updates the
// navigation target quickly. It does not wait for the
// slower RPM/PID cycle.
// This is important for a fast robot because waiting
// 250 ms before noticing a road edge would be far too
// slow at racing speed.
// ======================================================
void updateFastRoadControl() {
  unsigned long now = millis();
  if ( now - previousRoadControlTime < ROAD_CONTROL_INTERVAL_MS ) {
    return;
  }
  previousRoadControlTime = now;
```

ORIGINAL LINES 2,252-2,290

```
  updateWokwiBridgeScenario();
  updateVisionSerial();
  updateBridgePhase();
  // Internally rate-limited to 20 Hz.
  updateToF();
  readBoundarySensors();
  analyseRoadBoundaries();
  determineNavigationState();
  enforceValidNavigationState();
  handleStateChange();
  applyNavigationTargets();
```

Integrated autonomous controller

```
  if (isHardStopState()) {
    applyHardStopOutputs();
  }
}
// ====================================================
// SPEED RAMP UPDATE
// ====================================================
float rampTowardTarget( float currentValue, float requestedValue, float dt ) {
  if (dt <= 0.0f) return currentValue;
  float difference = requestedValue - currentValue;
  if (fabsf(difference) < 0.001f) return requestedValue;
  // Speed increases use the gentle acceleration rate.
  // Speed reductions use the faster deceleration rate.
  float rate = ( fabsf(requestedValue) > fabsf(currentValue) ) ? ACCEL_RAMP_RPM_PER_SEC :
    DECEL_RAMP_RPM_PER_SEC;
  float maximumStep = rate * dt;
  if (difference > maximumStep) return currentValue + maximumStep;
  if (difference < -maximumStep) return currentValue - maximumStep;
  return requestedValue;
}
void updateSpeedRamp( float dt ) {
  // Safety stops NEVER use a ramp.
  if (isHardStopState()) {
    leftRampedTargetRPM = 0.0f;
    rightRampedTargetRPM = 0.0f;
    return;
  }
```

Integrated autonomous controller

```
  leftRampedTargetRPM = rampTowardTarget( leftRampedTargetRPM, leftTargetRPM, dt );
```

ORIGINAL LINES 2,291-2,329

```
  rightRampedTargetRPM = rampTowardTarget( rightRampedTargetRPM, rightTargetRPM, dt );
}
// ======================================================
// RPM SAMPLE VALIDATION
// ======================================================
bool isValidRPMSample(float rpm) {
  return isfinite(rpm) && fabsf(rpm) <= MAX_VALID_RPM;
}
// ======================================================
// PID UPDATE
// ======================================================
void updatePID() {
  unsigned long now = millis();
  if ( now - previousPIDTime < PID_INTERVAL_MS ) {
    return;
  }
  float dt = ( now - previousPIDTime ) / 1000.0f;
  previousPIDTime = now;
  // RPM
  long leftDelta = leftEncoderCount - previousLeftPIDCount;
  long rightDelta = rightEncoderCount - previousRightPIDCount;
  previousLeftPIDCount = leftEncoderCount;
  previousRightPIDCount = rightEncoderCount;
  float newLeftRPM = ( (float)leftDelta / CPR ) * ( 60.0f / dt );
  float newRightRPM = ( (float)rightDelta / CPR ) * ( 60.0f / dt );
```

Integrated autonomous controller

```
  // Reject NaN, infinity, or an impossible Wokwi RPM spike.
  // If a bad sample occurs, keep the previous valid RPM so
  // one corrupt encoder sample cannot destabilise the PID.
  leftRPMValid = isValidRPMSample( newLeftRPM );
  rightRPMValid = isValidRPMSample( newRightRPM );
  if (leftRPMValid) {
    leftRawRPM = newLeftRPM;
    if (!leftRPMFilterInitialised) {
      leftRPM = newLeftRPM;
      leftRPMFilterInitialised = true;
    }
    else {
      leftRPM = RPM_FILTER_ALPHA * newLeftRPM + (1.0f - RPM_FILTER_ALPHA) * leftRPM;
    }
```

ORIGINAL LINES 2,330-2,368

```
  }
  if (rightRPMValid) {
    rightRawRPM = newRightRPM;
    if (!rightRPMFilterInitialised) {
      rightRPM = newRightRPM;
      rightRPMFilterInitialised = true;
    }
    else {
      rightRPM = RPM_FILTER_ALPHA * newRightRPM + (1.0f - RPM_FILTER_ALPHA) * rightRPM;
    }
  }
  // SLOWER HEALTH / STABILITY PERCEPTION
```

Integrated autonomous controller

```
2342    // Camera, IR and ToF are handled by the fast race loop.
2343    // This PID cycle updates current sensing and IMU data.
2344    updateINA219();
2345    imuValid = readGyroZ();
2346    if (!imuValid) {
2347      gyroZ = 0.0f;
2348      pitchDeg = 0.0f;
2349    }
2350    updateBridgePhase();
2351    updateSkidDetection();
2352    updateMotorStallDetection();
2353    // SAFETY / STABILITY DECISION
2354    // Re-run the state selector after the slower health
2355    // sensors update. Motor stall and skid states can then
2356    // override normal road driving.
2357    determineNavigationState();
2358    enforceValidNavigationState();
2359    handleStateChange();
2360    applyNavigationTargets();
2361    // Apply the physics-aware acceleration/deceleration ramp
2362    // before the PID calculates motor error.
2363    updateSpeedRamp( dt );
2364    // HARD STOP
2365    if (isHardStopState()) {
2366      leftPWM = 0;
2367      rightPWM = 0;
2368      leftIntegral = 0.0f;
```

ORIGINAL LINES 2,369-2,407

Integrated autonomous controller

```
    rightIntegral = 0.0f;
    digitalWrite(PWMA, LOW);
    digitalWrite(PWMB, LOW);
    return;
  }
  // ================================================
  // POSITIONAL PID + FEED-FORWARD + ANTI-WINDUP
  // ================================================
  // Previous controller:
  //   PWM = old PWM + correction
  // That could accumulate output and overshoot.
  // New controller:
  //   PWM = feed-forward + current PID correction
  // The output is recalculated fresh every PID cycle.
  // ================================================
  float leftError = leftRampedTargetRPM - leftRPM;
  float rightError = rightRampedTargetRPM - rightRPM;
  float leftDerivative = ( leftError - leftPreviousError ) / dt;
  float rightDerivative = ( rightError - rightPreviousError ) / dt;
  leftPreviousError = leftError;
  rightPreviousError = rightError;
  float leftCandidateIntegral = constrain( leftIntegral + leftError * dt, -PID_INTEGRAL_LIMIT, PID_INTEGRAL_LIMIT );
  float rightCandidateIntegral = constrain( rightIntegral + rightError * dt, -PID_INTEGRAL_LIMIT, PID_INTEGRAL_LIMIT );
  float leftFeedForward = leftRampedTargetRPM * PWM_FEEDFORWARD_PER_RPM;
  float rightFeedForward = rightRampedTargetRPM * PWM_FEEDFORWARD_PER_RPM;
  float leftCandidateOutput = leftFeedForward + Kp * leftError + Ki * leftCandidateIntegral + Kd * leftDerivative;
  float rightCandidateOutput = rightFeedForward + Kp * rightError + Ki * rightCandidateIntegral + Kd * rightDerivative;
```

Integrated autonomous controller

```
  bool allowLeftIntegral = ( ( leftCandidateOutput >= 0.0f && leftCandidateOutput <= 255.0f ) || ( leftCandidateOutput > 255.0f &&
        leftError < 0.0f ) || ( leftCandidateOutput < 0.0f && leftError > 0.0f ) );
  bool allowRightIntegral = ( ( rightCandidateOutput >= 0.0f && rightCandidateOutput <= 255.0f ) || ( rightCandidateOutput > 255.0f &&
        rightError < 0.0f ) || ( rightCandidateOutput < 0.0f && rightError > 0.0f ) );
  if (allowLeftIntegral) {
    leftIntegral = leftCandidateIntegral;
  }
  if (allowRightIntegral) {
    rightIntegral = rightCandidateIntegral;
  }
  if ( leftRPM > leftRampedTargetRPM + OVERSPEED_MARGIN_RPM && leftIntegral > 0.0f ) {
    leftIntegral *= OVERSPEED_INTEGRAL_BLEED;
```

ORIGINAL LINES 2,408-2,446

```
  }
  if ( rightRPM > rightRampedTargetRPM + OVERSPEED_MARGIN_RPM && rightIntegral > 0.0f ) {
    rightIntegral *= OVERSPEED_INTEGRAL_BLEED;
  }
  float leftPWMCommand = leftFeedForward + Kp * leftError + Ki * leftIntegral + Kd * leftDerivative;
  float rightPWMCommand = rightFeedForward + Kp * rightError + Ki * rightIntegral + Kd * rightDerivative;
  leftPWM = constrain( (int)roundf( leftPWMCommand ), 0, 255 );
  rightPWM = constrain( (int)roundf( rightPWMCommand ), 0, 255 );
}
// ======================================================
// NAVIGATION STATE VALIDATION / FAIL-SAFE
// ======================================================
// currentState should only contain one of the enum values
// assigned by determineNavigationState().
```

Integrated autonomous controller

```
// If an invalid value ever appears, fail SAFE:
//   invalid state -> CRITICAL STOP -> PWM 0 / 0
// ================================================
bool navigationStateIsValid() {
  switch (currentState) {
    case STATE_EMERGENCY_STOP:
    case STATE_VISION_DRIVE:
    case STATE_VISION_FALLBACK:
    case STATE_ROAD_DRIVE:
    case STATE_BOUNDARY_CORRECT_LEFT:
    case STATE_BOUNDARY_CORRECT_RIGHT:
    case STATE_BOUNDARY_CAUTION:
    case STATE_BOUNDARY_ESCAPE:
    case STATE_CAUTION:
    case STATE_AVOID_LEFT:
    case STATE_AVOID_RIGHT:
    case STATE_CRITICAL_AVOID_LEFT:
    case STATE_CRITICAL_AVOID_RIGHT:
    case STATE_CRITICAL_STOP:
    case STATE_NO_SAFE_PATH:
    case STATE_MOTOR_STALL:
    case STATE_SKID_CAUTION:
    case STATE_SENSOR_FAULT:
    case STATE_SYSTEM_OFF:
    case STATE_BRIDGE_APPROACH:
```

ORIGINAL LINES 2,447-2,485

```
    case STATE_BRIDGE_CLIMB:
```

Integrated autonomous controller

```
    case STATE_BRIDGE_CREST:
    case STATE_BRIDGE_DESCENT:
    case STATE_BRIDGE_EXIT:
      return true;
  }
  return false;
}
void enforceValidNavigationState() {
  if (navigationStateIsValid()) {
    return;
  }
  invalidStateRecoveryCount++;
  currentState = STATE_CRITICAL_STOP;
  leftTargetRPM = 0.0f;
  rightTargetRPM = 0.0f;
  leftRampedTargetRPM = 0.0f;
  rightRampedTargetRPM = 0.0f;
  leftPWM = 0;
  rightPWM = 0;
  leftIntegral = 0.0f;
  rightIntegral = 0.0f;
  digitalWrite(PWMA, LOW);
  digitalWrite(PWMB, LOW);
}
// ====================================================
// STATE NAME
// ====================================================
const char *stateName() {
```

Integrated autonomous controller

```
  switch (currentState) {
    case STATE_EMERGENCY_STOP:
      return "EMERGENCY STOP";
    case STATE_VISION_DRIVE:
      return "VISION PREDICTIVE DRIVE";
    case STATE_VISION_FALLBACK:
      return "VISION FALLBACK - IR ONLY";
    case STATE_ROAD_DRIVE:
      return "ROAD DRIVE";
    case STATE_BOUNDARY_CORRECT_LEFT:
```

ORIGINAL LINES 2,486-2,524

```
      return "BOUNDARY -> CORRECT LEFT";
    case STATE_BOUNDARY_CORRECT_RIGHT:
      return "BOUNDARY -> CORRECT RIGHT";
    case STATE_BOUNDARY_CAUTION:
      return "BOUNDARY CAUTION";
    case STATE_BOUNDARY_ESCAPE:
      return "BOUNDARY ESCAPE";
    case STATE_CAUTION:
      return "TOF CAUTION";
    case STATE_AVOID_LEFT:
      return "AVOID LEFT";
    case STATE_AVOID_RIGHT:
      return "AVOID RIGHT";
    case STATE_CRITICAL_AVOID_LEFT:
      return "CRITICAL AVOID LEFT";
    case STATE_CRITICAL_AVOID_RIGHT:
```

Integrated autonomous controller

```
        return "CRITICAL AVOID RIGHT";
      case STATE_CRITICAL_STOP:
        return "CRITICAL STOP";
      case STATE_NO_SAFE_PATH:
        return "NO SAFE PATH - STOP";
      case STATE_MOTOR_STALL:
        return "MOTOR STALL";
      case STATE_SKID_CAUTION:
        return "POSSIBLE SKID";
      case STATE_SENSOR_FAULT:
        return "TOF SENSOR FAULT - DEGRADED";
      case STATE_SYSTEM_OFF:
        return "SYSTEM OFF";
      case STATE_BRIDGE_APPROACH:
        return "BRIDGE APPROACH";
      case STATE_BRIDGE_CLIMB:
        return "BRIDGE CLIMB";
      case STATE_BRIDGE_CREST:
        return "BRIDGE CREST";
      case STATE_BRIDGE_DESCENT:
        return "BRIDGE DESCENT";
      case STATE_BRIDGE_EXIT:
        return "BRIDGE EXIT";
```

ORIGINAL LINES 2,525-2,562

```
    }
    return "INVALID STATE - FAILSAFE";
  }
```

Integrated autonomous controller

```
// ====================================================
// PRINT TTC
// ====================================================
void printTTCValue(float value) {
  if (value < 0.0f) {
    DebugSerial.print("N/A");
  }
  else {
    DebugSerial.print(value, 2);
    DebugSerial.print(" s");
  }
}
// ====================================================
// DIAGNOSTICS
// ====================================================
void printDiagnostics() {
  unsigned long now = millis();
  if ( now - previousPrintTime < PRINT_INTERVAL_MS ) {
    return;
  }
  previousPrintTime = now;
  DebugSerial.println();
  DebugSerial.println( "========================================" );
  DebugSerial.print( "RUN MODE          : " );
  DebugSerial.println( RUNNING_IN_WOKWI ? "WOKWI SCALED RPM" : ( PHYSICAL_INITIAL_TEST_PROFILE ? "PHYSICAL INITIAL TEST" : "PHYSICAL R
ACE" ) );
  DebugSerial.print( "NORMAL RPM LIMIT  : " );
  DebugSerial.println( NORMAL_BASE_RPM, 1 );
```

Integrated autonomous controller

```
  DebugSerial.print( "STATE             : " );
  DebugSerial.println( stateName() );
  DebugSerial.print( "STATE CODE        : " );
  DebugSerial.println( (int)currentState );
  DebugSerial.print( "STATE FAILSAFE CNT: " );
  DebugSerial.println( invalidStateRecoveryCount );
  DebugSerial.print( "E-STOP STATUS     : " );
  DebugSerial.println( emergencyStopActive ? "PRESSED" : "RELEASED" );
```

ORIGINAL LINES 2,563-2,601

```
  DebugSerial.print( "MAIN SWITCH       : " );
  DebugSerial.println( mainSwitchOn ? "ON" : "OFF" );
  DebugSerial.print( "START INTERLOCK   : " );
  if (raceStartInterlockCleared) {
    DebugSerial.println( "RACE ENABLED" );
  }
  else if (!mainSwitchOn) {
    DebugSerial.println( "READY - SWITCH ON TO START" );
  }
  else {
    DebugSerial.println( "LOCKED - SWITCH OFF FIRST" );
  }
  DebugSerial.print( "IR BOUNDARY       : " );
  for (int i = 0; i < 8; i++) {
    DebugSerial.print(sensor[i]);
  }
  DebugSerial.println();
  DebugSerial.print( "ROAD STATUS       : " );
```

Integrated autonomous controller

```
  if (!boundaryDetected) {
    DebugSerial.println( "CLEAR" );
  }
  else if ( centreBoundaryDetected && fabsf(boundarySteerError) <= 0.25f ) {
    DebugSerial.println( "CENTRE BOUNDARY / ESCAPE" );
  }
  else if ( boundarySteerError > 0.25f ) {
    DebugSerial.println( "LEFT BOUNDARY - STEER RIGHT" );
  }
  else if ( boundarySteerError < -0.25f ) {
    DebugSerial.println( "RIGHT BOUNDARY - STEER LEFT" );
  }
  else {
    DebugSerial.println( "BOTH SIDES / BALANCED" );
  }
  DebugSerial.print( "VISION SOURCE     : " );
  DebugSerial.println( USE_WOKWI_INTERNAL_VISION_SIM ? "WOKWI INTERNAL SIM" : "RASPBERRY PI UART" );
  DebugSerial.print( "VISION STATUS     : " );
  if (visionIsTrusted()) {
    DebugSerial.println( "ONLINE / TRUSTED" );
  }
```

ORIGINAL LINES 2,602-2,640

```
  else if (visionIsOnline()) {
    DebugSerial.println( "ONLINE / LOW CONFIDENCE" );
  }
  else {
    DebugSerial.println( "OFFLINE / STALE" );
```

Integrated autonomous controller

```
  }
  DebugSerial.print( "VISION STEER      : " );
  DebugSerial.print( visionSteerPct );
  DebugSerial.println( " %" );
  DebugSerial.print( "VISION SPEED      : " );
  DebugSerial.print( visionSpeedPct );
  DebugSerial.println( " %" );
  DebugSerial.print( "VISION CONFIDENCE : " );
  DebugSerial.print( visionConfidencePct );
  DebugSerial.println( " %" );
  DebugSerial.print( "BRIDGE CAM CONF   : " );
  DebugSerial.print( visionBridgeConfidencePct );
  DebugSerial.println( " %" );
  DebugSerial.print( "CORNER TEST       : " );
  DebugSerial.println( WOKWI_AUTO_CORNER_SPEED_TEST ? "ACTIVE / INDEPENDENT" : "DISABLED" );
  DebugSerial.print( "CORNER SEVERITY   : " );
  DebugSerial.print( visionCornerSeverityPct );
  DebugSerial.println( " %" );
  DebugSerial.print( "CORNER CLASS      : " );
  DebugSerial.println( cornerClassName() );
  DebugSerial.print( "CORNER RPM CAP    : " );
  DebugSerial.println( getVisionCornerSpeedCapRPM(), 2 );
  DebugSerial.print( "VISION BASE RPM   : " );
  DebugSerial.println( getVisionBaseRPM(), 2 );
  DebugSerial.print( "PI RX BYTES       : " );
  DebugSerial.println( piRxByteCount );
  DebugSerial.print( "PI GOOD PACKETS   : " );
  DebugSerial.println( piGoodPacketCount );
```

Integrated autonomous controller

```
  DebugSerial.print( "PI BAD PACKETS    : " );
  DebugSerial.println( piBadPacketCount );
  DebugSerial.print( "PI LAST RAW       : " );
  DebugSerial.println( piLastRawPacket );
  DebugSerial.print( "BOUNDARY STEER    : " );
  DebugSerial.println( boundarySteerError, 2 );
```

ORIGINAL LINES 2,641-2,679

```
  DebugSerial.print( "LAST BOUNDARY     : " );
  DebugSerial.println( lastBoundarySteerError, 2 );
  DebugSerial.print( "BOUNDARY SEVERITY : " );
  DebugSerial.println( boundarySeverity, 1 );
  DebugSerial.println();
  DebugSerial.print( "TOF SOURCE        : " );
  if (USE_WOKWI_INTERNAL_TOF_SIM) {
    DebugSerial.println( "WOKWI INTERNAL" );
  }
  else if (USE_WOKWI_TOF_SLIDER_CHIPS) {
    DebugSerial.println( "VL53L1X SLIDERS" );
  }
  else {
    DebugSerial.println( "PHYSICAL I2C" );
  }
  DebugSerial.print( "IMU SOURCE        : " );
  DebugSerial.println( USE_WOKWI_INTERNAL_IMU_SIM ? "WOKWI INTERNAL" : "PHYSICAL I2C" );
  DebugSerial.print( "INA SOURCE        : " );
  if (USE_WOKWI_INTERNAL_INA_SIM) {
    DebugSerial.println( "WOKWI INTERNAL" );
```

Integrated autonomous controller

```
2661    }
2662    else if (USE_WOKWI_INA219_SLIDER_CHIP) {
2663      DebugSerial.println( "INA219 SLIDERS" );
2664    }
2665    else {
2666      DebugSerial.println( "PHYSICAL I2C" );
2667    }
2668    DebugSerial.print( "LEFT TOF          : " );
2669    if (tofValid) {
2670      DebugSerial.print(leftDistance);
2671      DebugSerial.println(" mm");
2672    }
2673    else {
2674      DebugSerial.println("INVALID");
2675    }
2676    DebugSerial.print( "RIGHT TOF         : " );
2677    if (tofValid) {
2678      DebugSerial.print(rightDistance);
2679      DebugSerial.println(" mm");
```

ORIGINAL LINES 2,680-2,718

```
2680    }
2681    else {
2682      DebugSerial.println("INVALID");
2683    }
2684    DebugSerial.print( "LEFT CLOSING      : " );
2685    DebugSerial.print(leftClosingSpeed, 1);
2686    DebugSerial.println(" mm/s");
```

Integrated autonomous controller

```
2687    DebugSerial.print( "RIGHT CLOSING     : " );
2688    DebugSerial.print(rightClosingSpeed, 1);
2689    DebugSerial.println(" mm/s");
2690    DebugSerial.print( "LEFT TTC          : " );
2691    printTTCValue(leftTTC);
2692    DebugSerial.println();
2693    DebugSerial.print( "RIGHT TTC         : " );
2694    printTTCValue(rightTTC);
2695    DebugSerial.println();
2696    DebugSerial.println();
2697    DebugSerial.print( "TRACK MODE        : " );
2698    DebugSerial.println( "CAMERA + IR + TOF RACING" );
2699    DebugSerial.print( "OBSTACLE MODE     : " );
2700    DebugSerial.println( OBSTACLE_AVOIDANCE_ENABLED ? "ENABLED" : "ENABLED - CARS / WALLS" );
2701    DebugSerial.print( "ROAD LOOP         : " );
2702    DebugSerial.print( 1000UL / ROAD_CONTROL_INTERVAL_MS );
2703    DebugSerial.println( " Hz" );
2704    DebugSerial.println();
2705    DebugSerial.print( "INA219 STATUS     : " );
2706    DebugSerial.println( inaValid ? "ONLINE" : "READ ERROR" );
2707    DebugSerial.print( "BUS VOLTAGE       : " );
2708    if (inaValid) {
2709      DebugSerial.print( busVoltageV, 2 );
2710      DebugSerial.println( " V" );
2711    }
2712    else {
2713      DebugSerial.println( "N/A" );
2714    }
```

Integrated autonomous controller

```
2715    DebugSerial.print( "MOTOR CURRENT     : " );
2716    if (inaValid) {
2717      DebugSerial.print( motorCurrentMA, 0 );
2718      DebugSerial.println( " mA" );
```

ORIGINAL LINES 2,719-2,757

```
2719    }
2720    else {
2721      DebugSerial.println( "N/A" );
2722    }
2723    DebugSerial.print( "CURRENT STATUS    : " );
2724    DebugSerial.println( currentStatusName() );
2725    DebugSerial.print( "STALL FLAG        : " );
2726    DebugSerial.println( motorStallDetected ? "YES" : "NO" );
2727    DebugSerial.print( "STALL LATCH       : " );
2728    DebugSerial.println( motorStallLatched ? "LOCKED" : "CLEAR" );
2729    DebugSerial.print( "STALL SIDE        : " );
2730    DebugSerial.println( stallSideName() );
2731    DebugSerial.print( "STALL RESET       : " );
2732    DebugSerial.println( motorStallLatched ? "MAIN SWITCH OFF -> ON REQUIRED" : "READY" );
2733    DebugSerial.println();
2734    DebugSerial.print( "MPU STATUS        : " );
2735    DebugSerial.println( imuValid ? "ONLINE" : "READ ERROR" );
2736    DebugSerial.print( "GYRO Z / YAW      : " );
2737    DebugSerial.print( gyroZ, 2 );
2738    DebugSerial.println( " deg/s" );
2739    DebugSerial.print( "PITCH            : " );
2740    DebugSerial.print( pitchDeg, 2 );
```

Integrated autonomous controller

```
  DebugSerial.println( " deg" );
  DebugSerial.print( "BRIDGE PHASE      : " );
  DebugSerial.println( bridgePhaseName() );
  DebugSerial.print( "SKID FLAG         : " );
  DebugSerial.println( possibleSkid ? "YES" : "NO" );
  DebugSerial.println();
  DebugSerial.print( "LEFT REQUEST RPM  : " );
  DebugSerial.println( leftTargetRPM, 2 );
  DebugSerial.print( "RIGHT REQUEST RPM : " );
  DebugSerial.println( rightTargetRPM, 2 );
  DebugSerial.print( "LEFT PID SETPOINT : " );
  DebugSerial.println( leftRampedTargetRPM, 2 );
  DebugSerial.print( "RIGHT PID SETPOINT: " );
  DebugSerial.println( rightRampedTargetRPM, 2 );
  DebugSerial.print( "LEFT ACTUAL RPM   : " );
  if (leftRPMValid) {
    DebugSerial.println( leftRPM, 2 );
```

ORIGINAL LINES 2,758-2,796

```
  }
  else {
    DebugSerial.println( "INVALID" );
  }
  DebugSerial.print( "RIGHT ACTUAL RPM  : " );
  if (rightRPMValid) {
    DebugSerial.println( rightRPM, 2 );
  }
  else {
```

Integrated autonomous controller

```
    DebugSerial.println( "INVALID" );
  }
  DebugSerial.print( "LEFT RAW RPM      : " );
  DebugSerial.println( leftRawRPM, 2 );
  DebugSerial.print( "RIGHT RAW RPM     : " );
  DebugSerial.println( rightRawRPM, 2 );
  DebugSerial.print( "RPM SAMPLE STATUS : " );
  DebugSerial.println( (leftRPMValid && rightRPMValid) ? "OK" : "CHECK ENCODERS" );
  DebugSerial.println( "PID MODE          : POSITIONAL + FF + ANTI-WINDUP" );
  DebugSerial.print( "LEFT PWM          : " );
  DebugSerial.println(leftPWM);
  DebugSerial.print( "RIGHT PWM         : " );
  DebugSerial.println(rightPWM);
  // Ensure the complete diagnostics frame leaves the UART buffer
  DebugSerial.flush();
}
// ====================================================
// SETUP
// ====================================================
void setup() {
  DebugSerial.begin(115200);
  PiSerial.begin(115200);
  // Allow the first ToF update during setup.
  previousToFTime = millis() - TOF_INTERVAL_MS;
  delay(500);
  // IMPORTANT:
  // Print immediately so we know setup has started.
  DebugSerial.println();
```

Integrated autonomous controller

```
2795    DebugSerial.println( "=======================================" );
2796    DebugSerial.println( " ROBO RUMBLE - FULL SENSOR RACE CONTROLLER" );
```

ORIGINAL LINES 2,797-2,835

```
2797    DebugSerial.println( "=======================================" );
2798    DebugSerial.println();
2799    DebugSerial.println( "[BOOT] Setup started" );
2800    DebugSerial.println( "[BOOT] Pi vision UART: PB11 RX / PB10 TX @ 115200" );
2801    DebugSerial.print( "[BOOT] Vision source: " );
2802    DebugSerial.println( USE_WOKWI_INTERNAL_VISION_SIM ? "WOKWI INTERNAL SIM" : "RASPBERRY PI UART" );
2803    DebugSerial.println( "[BOOT] Pi RX diagnostics enabled" );
2804    DebugSerial.println( "[BOOT] FINAL INTEGRATED MODE: scripted tests DISABLED" );
2805    DebugSerial.println( "[BOOT] STARTUP INTERLOCK: MAIN must be OFF, then switched ON" );
2806    DebugSerial.println( "[BOOT] MAIN SWITCH: 60 ms DEBOUNCE + ONE-SHOT RACE START" );
2807    DebugSerial.println( "[BOOT] Sensor sources:" );
2808    DebugSerial.print( "       ToF : " );
2809    if (USE_WOKWI_INTERNAL_TOF_SIM) {
2810      DebugSerial.println( "WOKWI INTERNAL" );
2811    }
2812    else if (USE_WOKWI_TOF_SLIDER_CHIPS) {
2813      DebugSerial.println( "VL53L1X SLIDERS" );
2814    }
2815    else {
2816      DebugSerial.println( "PHYSICAL I2C" );
2817    }
2818    DebugSerial.print( "       IMU : " );
2819    DebugSerial.println( USE_WOKWI_INTERNAL_IMU_SIM ? "WOKWI INTERNAL" : "PHYSICAL I2C" );
2820    DebugSerial.print( "       INA : " );
```

Integrated autonomous controller

```
2821    DebugSerial.println( USE_WOKWI_INTERNAL_INA_SIM ? "WOKWI INTERNAL" : "PHYSICAL I2C" );
2822    // EMERGENCY STOP
2823    pinMode( ESTOP_PIN, INPUT_PULLUP );
2824    readEmergencyStop();
2825    // MOTOR PINS
2826    pinMode(PWMA, OUTPUT);
2827    pinMode(AIN1, OUTPUT);
2828    pinMode(AIN2, OUTPUT);
2829    pinMode(PWMB, OUTPUT);
2830    pinMode(BIN1, OUTPUT);
2831    pinMode(BIN2, OUTPUT);
2832    pinMode(STBY, OUTPUT);
2833    // SAFE BOOT OUTPUTS:
2834    // Before any sensor initialisation or race logic, both PWM
2835    // outputs and the motor driver are forced OFF.
```

ORIGINAL LINES 2,836-2,874

```
2836    digitalWrite(PWMA, LOW);
2837    digitalWrite(PWMB, LOW);
2838    digitalWrite(STBY, LOW);
2839    // ENCODERS
2840    pinMode(LEFT_ENC_A, INPUT);
2841    pinMode(LEFT_ENC_B, INPUT);
2842    pinMode(RIGHT_ENC_A, INPUT);
2843    pinMode(RIGHT_ENC_B, INPUT);
2844    // IR
2845    for (int i = 0; i < 8; i++) {
2846      pinMode( IR_PINS[i], INPUT );
```

Integrated autonomous controller

```
2847    }
2848    // TOF XSHUT
2849    // PB3/PB4 are used as GPIO for the two ToF shutdown pins.
2850    // On physical STM32F103 hardware these pins overlap with
2851    // the default JTAG interface. If supported by the core,
2852    // disable JTAG while retaining SWD.
2853    // For this Wokwi model the XSHUT lines stay HIGH because
2854    // the two simulated sensors already have unique addresses.
2855    // The race-safe custom ToF chip deliberately ignores live
2856    // XSHUT disconnects to avoid a Wokwi custom-I2C bus lockup.
2857    #ifdef __HAL_AFIO_REMAP_SWJ_NOJTAG
2858      __HAL_RCC_AFIO_CLK_ENABLE();
2859      __HAL_AFIO_REMAP_SWJ_NOJTAG();
2860    #endif
2861    // MAIN ON/OFF SWITCH
2862    // PA15 is usable as GPIO after JTAG is disabled.
2863    pinMode( MAIN_SWITCH_PIN, INPUT_PULLUP );
2864    // Initialise the debounced MAIN-switch state.
2865    // This NEVER starts the race at boot, even if the switch is ON.
2866    initialiseMainSwitch();
2867    pinMode( LEFT_XSHUT, OUTPUT );
2868    pinMode( RIGHT_XSHUT, OUTPUT );
2869    // ================================================
2870    // WOKWI-SAFE XSHUT INITIALISATION
2871    // ================================================
2872    // IMPORTANT:
2873    // Our two custom Wokwi VL53L1X chips already have fixed
```

Integrated autonomous controller

```
2874    // simulation addresses (0x30 and 0x31).
```

ORIGINAL LINES 2,875-2,913

```
2875    // Toggling a custom I2C device off and back on with
2876    // XSHUT can make Wokwi's shared custom-I2C bus hang.
2877    // Therefore, in Wokwi we simply keep both sensors
2878    // enabled from startup.
2879    // The PHYSICAL robot is different: real VL53L1X
2880    // sensors normally start at the same default address,
2881    // so physical firmware will later sequence XSHUT and
2882    // assign unique I2C addresses during startup.
2883    // ================================================
2884    digitalWrite( LEFT_XSHUT, HIGH );
2885    digitalWrite( RIGHT_XSHUT, HIGH );
2886    DebugSerial.println( "[BOOT] GPIO configured" );
2887    // I2C
2888    // The bus remains configured even when the integrated
2889    // Wokwi internal-sensor mode is enabled. This preserves
2890    // the physical pin map in one codebase.
2891    Wire.setSDA(PB7);
2892    Wire.setSCL(PB6);
2893    Wire.begin();
2894    Wire.setClock( 100000 );
2895    DebugSerial.println( "[BOOT] I2C started" );
2896    delay(100);
2897    // TOF
2898    // Do not perform a VL53L1X transaction inside setup().
2899    // The custom Wokwi I2C pair is serviced by the normal
```

Integrated autonomous controller

```
  // race loop after all devices have finished booting.
  // This also keeps startup deterministic: the Serial
  // Monitor reaches READY before the first ToF sample.
  DebugSerial.println( "[BOOT] VL53L1X pair: DEFERRED TO RACE LOOP" );
  // MPU
  DebugSerial.println( "[BOOT] Initialising MPU6050..." );
  imuValid = initialiseMPU();
  DebugSerial.print( "[BOOT] MPU6050: " );
  DebugSerial.println( imuValid ? "ONLINE" : "FAILED" );
  // INA219
  DebugSerial.println( "[BOOT] Checking INA219..." );
  updateINA219( true );
  DebugSerial.print( "[BOOT] INA219: " );
  DebugSerial.println( inaValid ? "ONLINE" : "FAILED" );
```

ORIGINAL LINES 2,914-2,951

```
  // ENCODER INITIAL STATES
  previousLeftEncoderState = ( digitalRead(LEFT_ENC_A) << 1 ) | digitalRead(LEFT_ENC_B);
  previousRightEncoderState = ( digitalRead(RIGHT_ENC_A) << 1 ) | digitalRead(RIGHT_ENC_B);
  DebugSerial.println( "[BOOT] Encoders configured" );
  // MOTORS
  motorsForward();
  if (emergencyStopActive) {
    applyEmergencyStopImmediately();
    DebugSerial.println( "[BOOT] E-STOP: PRESSED - MOTOR DRIVER DISABLED" );
  }
  else {
    DebugSerial.println( "[BOOT] E-STOP: RELEASED" );
```

Integrated autonomous controller

```
2926    }
2927    DebugSerial.print( "[BOOT] MAIN SWITCH: " );
2928    DebugSerial.println( mainSwitchOn ? "ON" : "OFF" );
2929    pwmPeriodStart = micros();
2930    previousPIDTime = millis();
2931    previousRoadControlTime = millis();
2932    previousToFTime = millis() - TOF_INTERVAL_MS;
2933    previousINATime = millis() - INA_INTERVAL_MS;
2934    DebugSerial.println( "[BOOT] Motor driver enabled" );
2935    DebugSerial.print( "[BOOT] Speed ramp: ACCEL " );
2936    DebugSerial.print( ACCEL_RAMP_RPM_PER_SEC, 1 );
2937    DebugSerial.print( " RPM/s | DECEL " );
2938    DebugSerial.print( DECEL_RAMP_RPM_PER_SEC, 1 );
2939    DebugSerial.println( " RPM/s" );
2940    DebugSerial.println( "[BOOT] PID: POSITIONAL + FEED-FORWARD + ANTI-WINDUP" );
2941    DebugSerial.println( "[BOOT] Stall latch: MANUAL MAIN OFF -> ON RESET" );
2942    DebugSerial.println( RUNNING_IN_WOKWI ? "[BOOT] INA219 source: INTERACTIVE WOKWI SLIDERS" : "[BOOT] INA219 source: PHYSICAL I2C" );
2943    DebugSerial.print( "[BOOT] Run profile: " );
2944    DebugSerial.println( RUNNING_IN_WOKWI ? "WOKWI SCALED RPM" : ( PHYSICAL_INITIAL_TEST_PROFILE ? "PHYSICAL INITIAL TEST" : "PHYSICAL R
ACE" ) );
2945    DebugSerial.print( "[BOOT] Normal target RPM: " );
2946    DebugSerial.println( NORMAL_BASE_RPM, 1 );
2947    DebugSerial.print( "[BOOT] Dynamic corner-speed test: " );
2948    DebugSerial.println( WOKWI_AUTO_CORNER_SPEED_TEST ? "ENABLED" : "DISABLED" );
2949    DebugSerial.println( "[BOOT] Wokwi test scenario: CORNER SPEED ONLY" );
2950    DebugSerial.print( "[BOOT] Autonomous bridge test: " );
2951    DebugSerial.println( WOKWI_AUTO_BRIDGE_TEST ? "ENABLED" : "DISABLED" );
```

ORIGINAL LINES 2,952-2,990

Integrated autonomous controller

```
  DebugSerial.println( "[BOOT] Bridge profile: MILD SLOPE / HIGHER SPEED" );
  DebugSerial.println();
  DebugSerial.println( "[READY] Starting autonomous control..." );
  DebugSerial.println();
}
// ======================================================
// LOOP
// ======================================================
void loop() {
  // ======================================================
  // EMERGENCY STOP IS CHECKED EVERY LOOP
  // ======================================================
  readEmergencyStop();
  readMainSwitch();
  if (emergencyStopActive) {
    applyEmergencyStopImmediately();
    // Keep sampling encoders so diagnostics can show
    // the wheels winding down after the drive is cut.
    readLeftEncoder();
    readRightEncoder();
    printDiagnostics();
    return;
  }
  if ( !mainSwitchOn || !raceStartInterlockCleared ) {
    applySystemOffImmediately();
    // In Wokwi the MCU stays alive so we can display
    // SYSTEM OFF diagnostics.
```

Integrated autonomous controller

```
    readLeftEncoder();
    readRightEncoder();
    printDiagnostics();
    return;
  }
  // E-Stop released + main switch ON:
  digitalWrite( STBY, HIGH );
  serviceMotors();
  updateWokwiBridgeScenario();
  // Read Raspberry Pi look-ahead commands as soon as possible.
  updateVisionSerial();
  // Fast steering / road-edge reaction
```

ORIGINAL LINES 2,991-2,995

```
  updateFastRoadControl();
  // Slower wheel-speed PID + health/stability checks
  updatePID();
  printDiagnostics();
```

Integrated autonomous controller

```
}
```